

Day 2 – 1,2



# Self-supervised learning

Jonghyun Choi

Associate Professor

Department of Electrical and Computer Engineering  
Seoul National University



# Schedule

Each hour is scheduled as **50 min lecture + 10 min break**

- **Day 1:** 4 hr lecture + 4 hr practice
  - **AM:** CNN, Transformers, Object detection
  - **PM:** DETR, MI-DETR
- **Day 2:** 4 hr lecture + 4 hr practice
  - **AM:** Self-supervised learning, Weakly supervised learning, Multimodal models, Semantic segmentation
  - **PM:** DINO, SAM variants

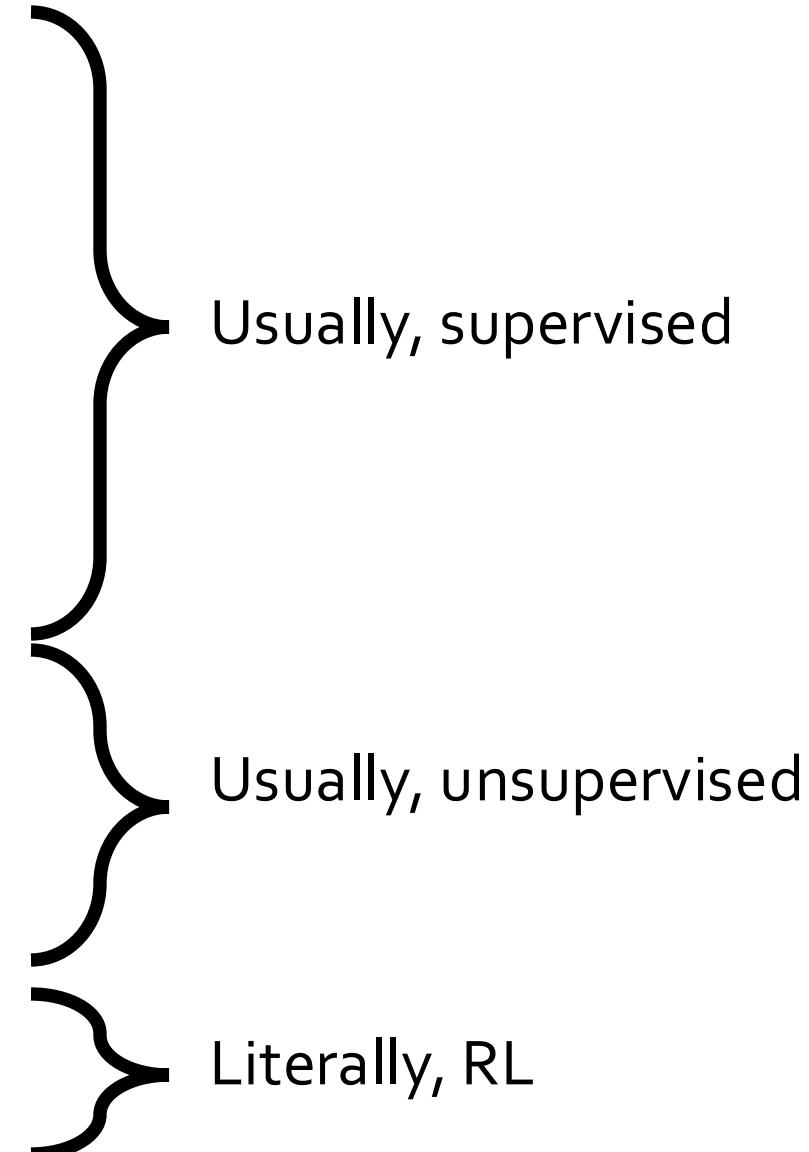
# Machine learning problems

- Classification
  - Data to discrete class label
  - Predicting a class label
- Regression
  - Predicting a numeric value
- Similarity
  - Finding similar/dissimilar data
- Clustering
  - Discovering structure in data
- Embedding
  - Data to a vector
- Reinforcement Learning
  - Training by feedback

# Categorizing machine learning methods

- Supervised learning
  - Train a model with data with label
- Unsupervised learning
  - Train a model with data without label
- Semi-supervised learning
  - Train a model with data with label for some
- Active learning
  - Train a model with selected data with label
- Reinforcement learning
  - Train a model with indirect label

# Machine learning problems

- Classification
    - Data to discrete class label
    - Predicting a class label
  - Regression
    - Predicting a numeric value
  - Similarity
    - Finding similar/dissimilar data
  - Clustering
    - Discovering structure in data
  - Embedding
    - Data to a vector
  - Reinforcement Learning
    - Training by feedback
- 
- The diagram uses three large curly braces on the right side to group the machine learning problems into three categories:
- Usually, supervised**: This group includes Classification, Regression, and Similarity.
  - Usually, unsupervised**: This group includes Clustering and Embedding.
  - Literally, RL**: This group includes Reinforcement Learning.

Self-supervised learning  
is a type of unsupervised learning

# What can we learn from unlabeled data?

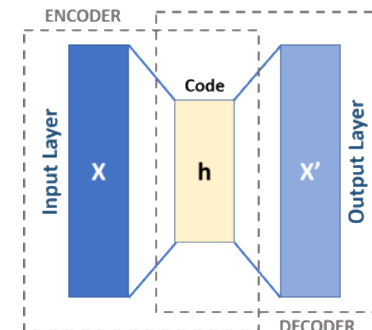
- Innate structure of the data
- What innate structure?
  - Automatically learned?

There is no magic!

# Enforcing implicit loss

- Optimizing on implicit criteria
  - Clustering
    - Group points closer than the other groups
  - Anomaly detection
    - Detect something that is not normal
  - Representation learning
    - **Dimension reduction:** obtain useful (a small set of) dimensions
    - **Auto Encoder:** Encoding large dimensional data into a compact representation by learning an encoder-decoder pair to reproduce the input

An Example Auto Encoder





# What is representation learning

- Our neural network takes the raw-input (i.e., not transformed by a feature transformation) and results in output
  - i.e., End-to-end network
- The feature (the representation) of the raw-input is learned by the gradient training
  - Called representation learning

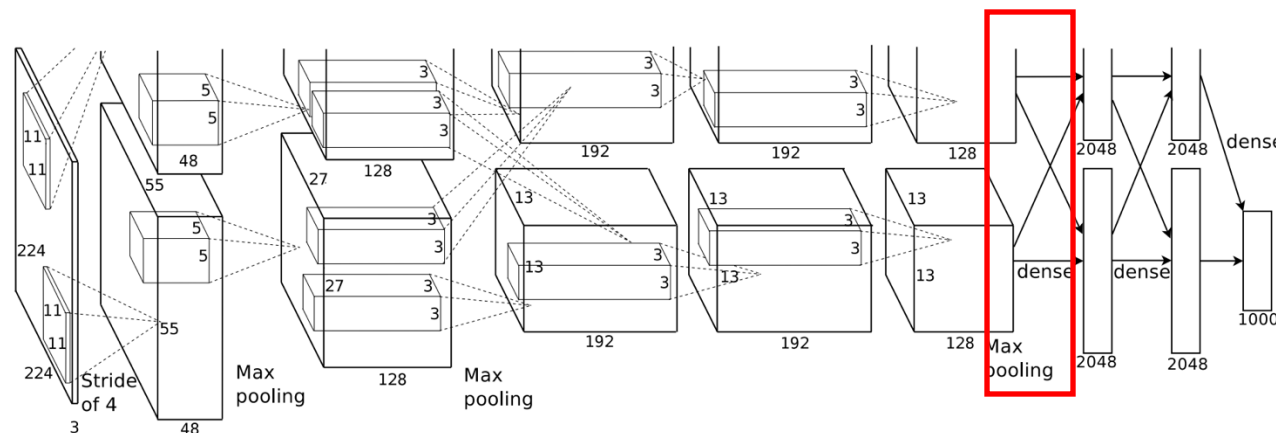


Figure 2: An illustration of the architecture of our CNN, explicitly showing the delineation of responsibilities between the two GPUs. One GPU runs the layer-parts at the top of the figure while the other runs the layer-parts at the bottom. The GPUs communicate only at certain layers. The network's input is 150,528-dimensional, and the number of neurons in the network's remaining layers is given by 253,440–186,624–64,896–64,896–43,264–4,096–4,096–1000.

# Representation learning can be done

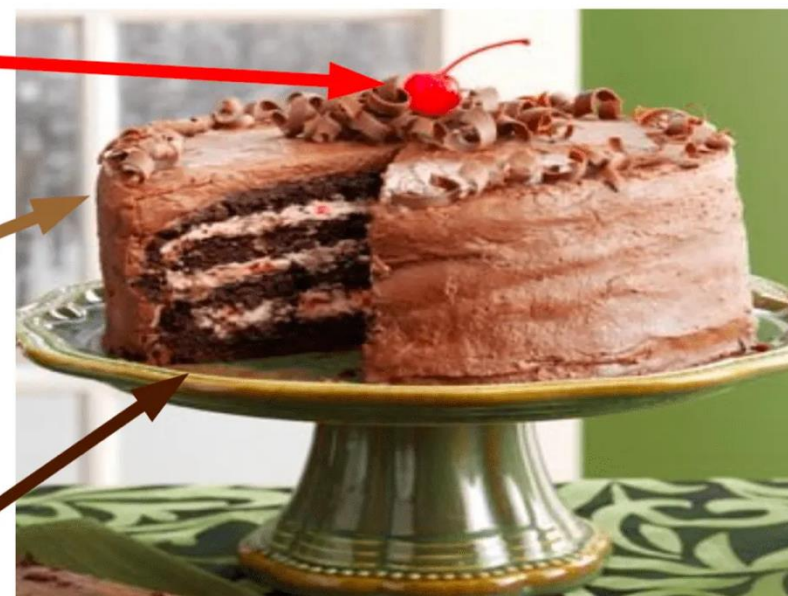
- In supervised learning manner
  - Feature extracting (sub-)network
- In unsupervised learning manner (more widely used case)
  - Auto-encoder
  - Self-supervised learning
    - ✓ Contrastive learning
- As a **pretraining** scheme
  - To warm up the network with better initial points
  - The defined unsupervised task is called **pretext task**
    - ✓ As opposed to the pretext task, the **task of interest** is called **downstream task**

# Yann LeCun's Cake Analogy

Y. LeCun

## How Much Information is the Machine Given during Learning?

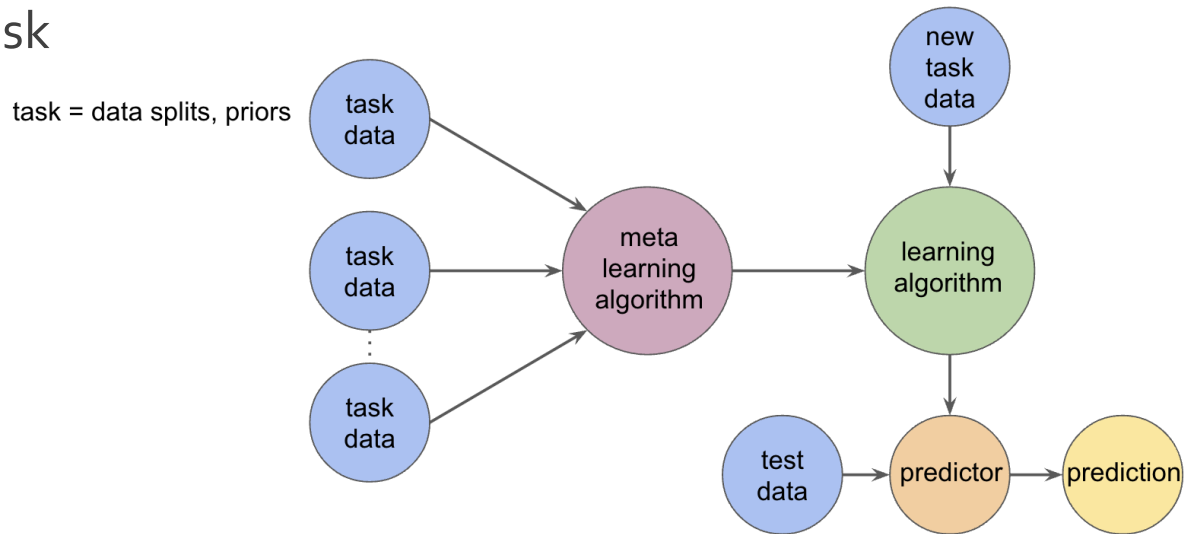
- ▶ **“Pure” Reinforcement Learning (cherry)**
  - ▶ The machine predicts a scalar reward given once in a while.
  - ▶ **A few bits for some samples**
- ▶ **Supervised Learning (icing)**
  - ▶ The machine predicts a category or a few numbers for each input
  - ▶ Predicting human-supplied data
  - ▶ **10→10,000 bits per sample**
- ▶ **Self-Supervised Learning (cake génoise)**
  - ▶ The machine predicts any part of its input for any observed part.
  - ▶ Predicts future frames in videos
  - ▶ **Millions of bits per sample**



# How to use the pretrained models for downstream tasks

- Fine-tuning
  - Use the pretrained model as initial parameters
  - Train the network on the downstream task
    - ✓ Using the dataset of the downstream task
    - ✓ Train the network in multiple epochs

- Meta learning



# 'Learning paradigms' relevant to the Meta-learning

- Transfer learning
  - Transfer the knowledge trained for task A to task B
- Domain adaptation
  - **Domain:** The set that generates input  $x$ 
    - ✓ E.g., English language / Korean language
    - ✓ E.g., Images collected by a phone / Images collected by a DSL camera
    - ✓ E.g., data in dataset A / data in dataset B
  - **Domain adaptation:** Adapt model trained in Domain A to perform well in Domain B

# Supervised representation learning in AlexNet

Representation learning part (Feature extracting sub-network)

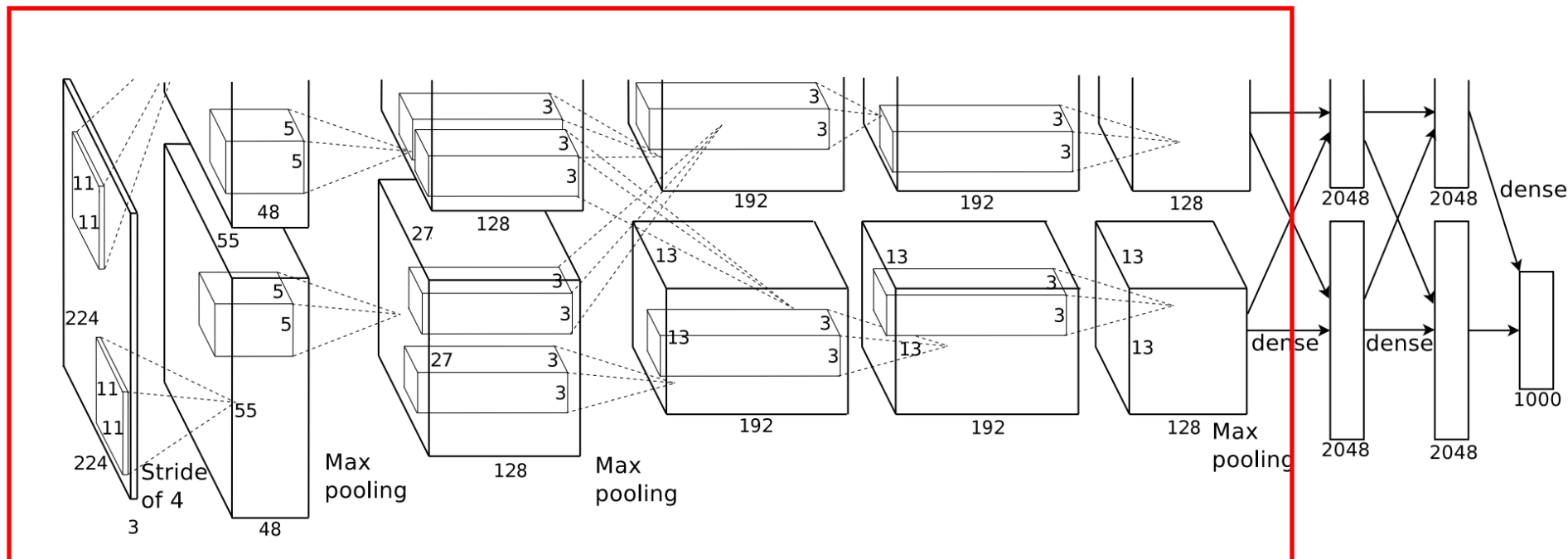
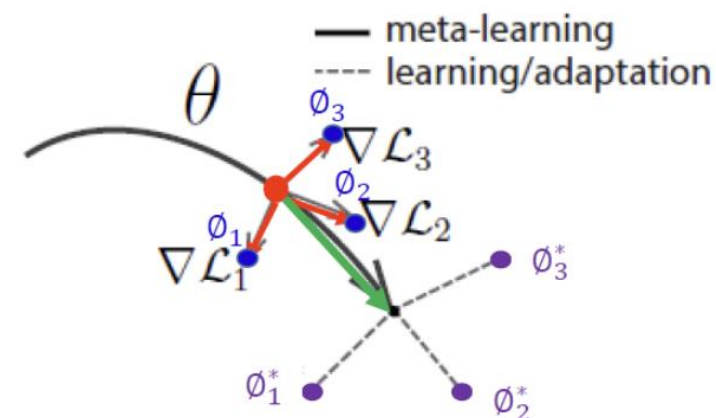


Figure 2: An illustration of the architecture of our CNN, explicitly showing the delineation of responsibilities between the two GPUs. One GPU runs the layer-parts at the top of the figure while the other runs the layer-parts at the bottom. The GPUs communicate only at certain layers. The network's input is 150,528-dimensional, and the number of neurons in the network's remaining layers is given by 253,440–186,624–64,896–64,896–43,264–4096–4096–1000.

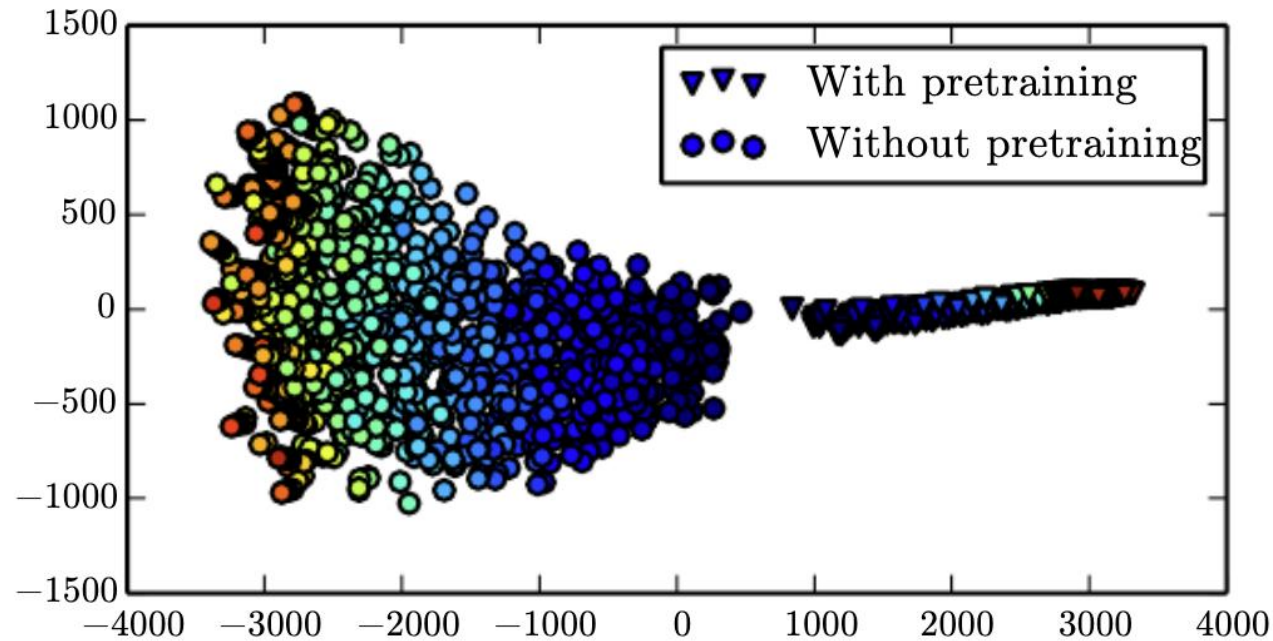
# Model agnostic meta-learning (MAML)

## Two keywords

- Model Agnostic: Applicable to all gradient based training models
- Fast adaptation: With a very few adaptation process, we can utilize the meta-learned model to a downstream task
- Idea: Pretraining with many tasks
  - Similar to instruction fine-tuning



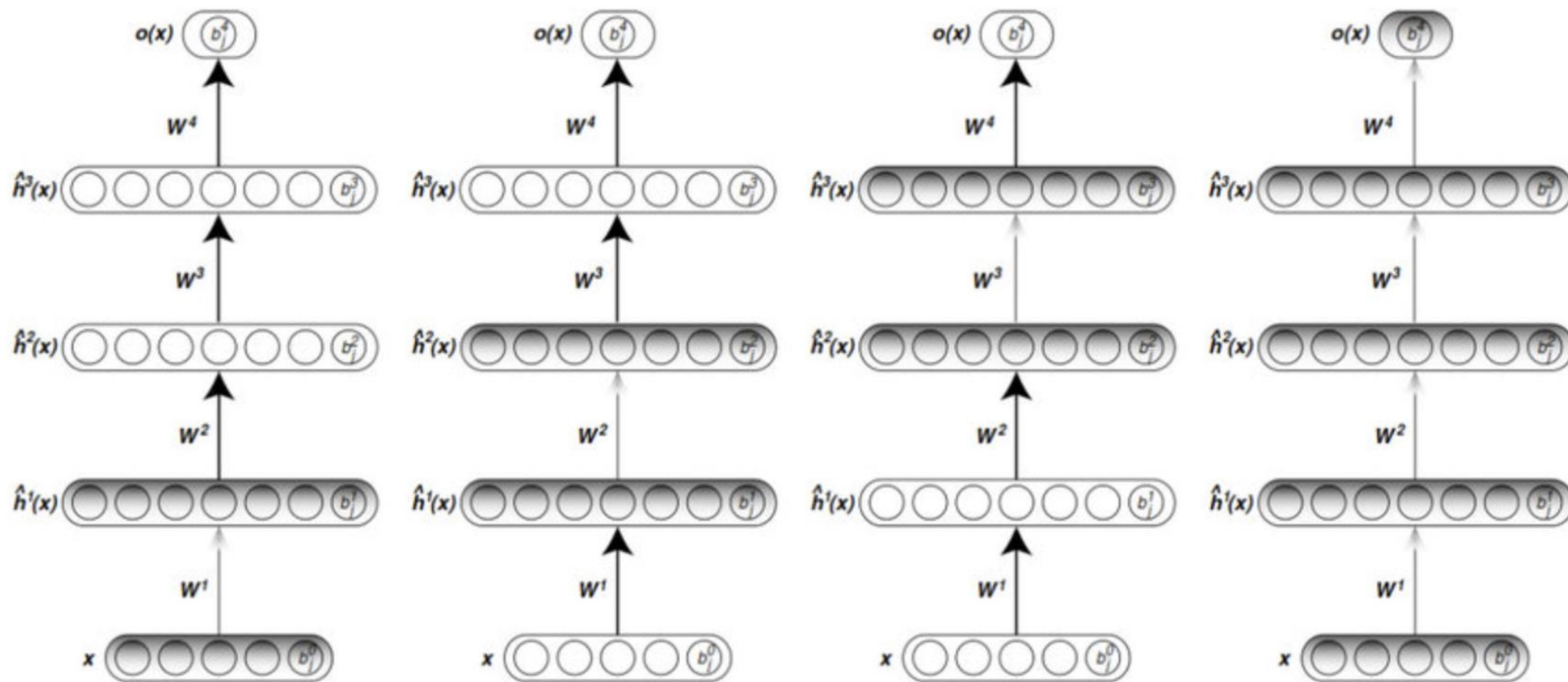
# Effect of pretraining



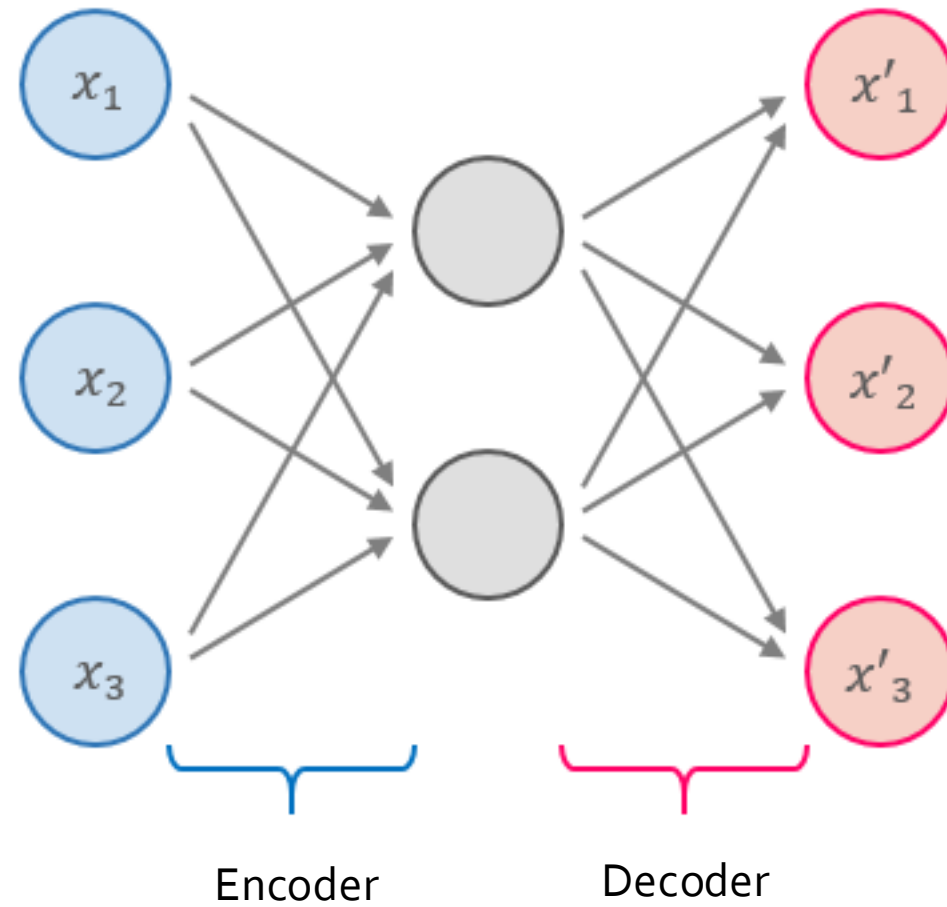
- Visualization via nonlinear projection of learning trajectories of different neural networks
- Color: time



# Greedy Layer-Wise Unsupervised Pretraining



# Auto encoder



# Auto encoder

7

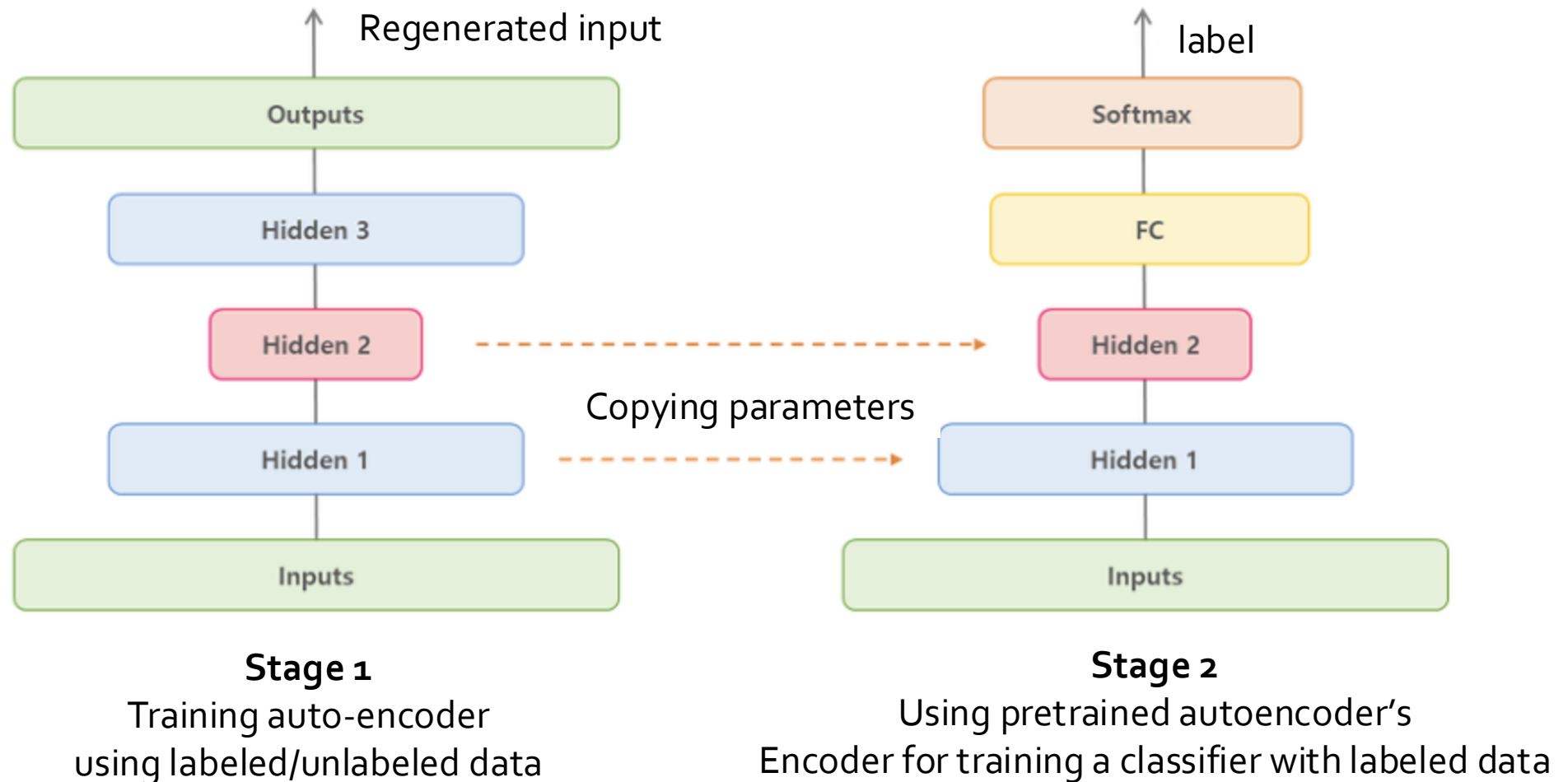
2

→  
Autoencoder

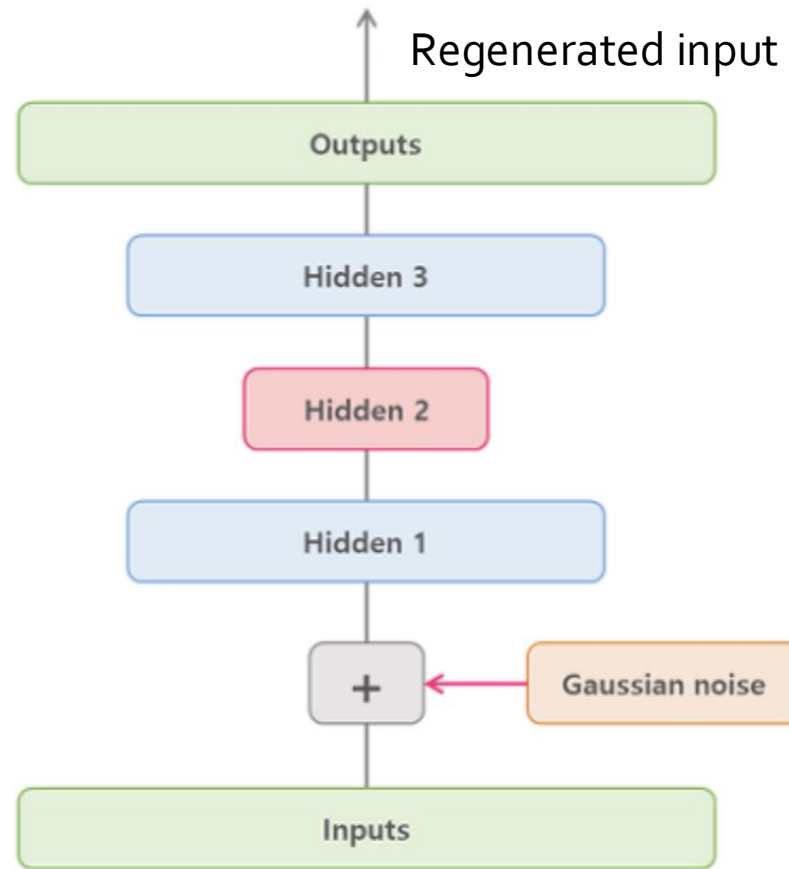
7

2

# Auto encoder for unsupervised pretraining

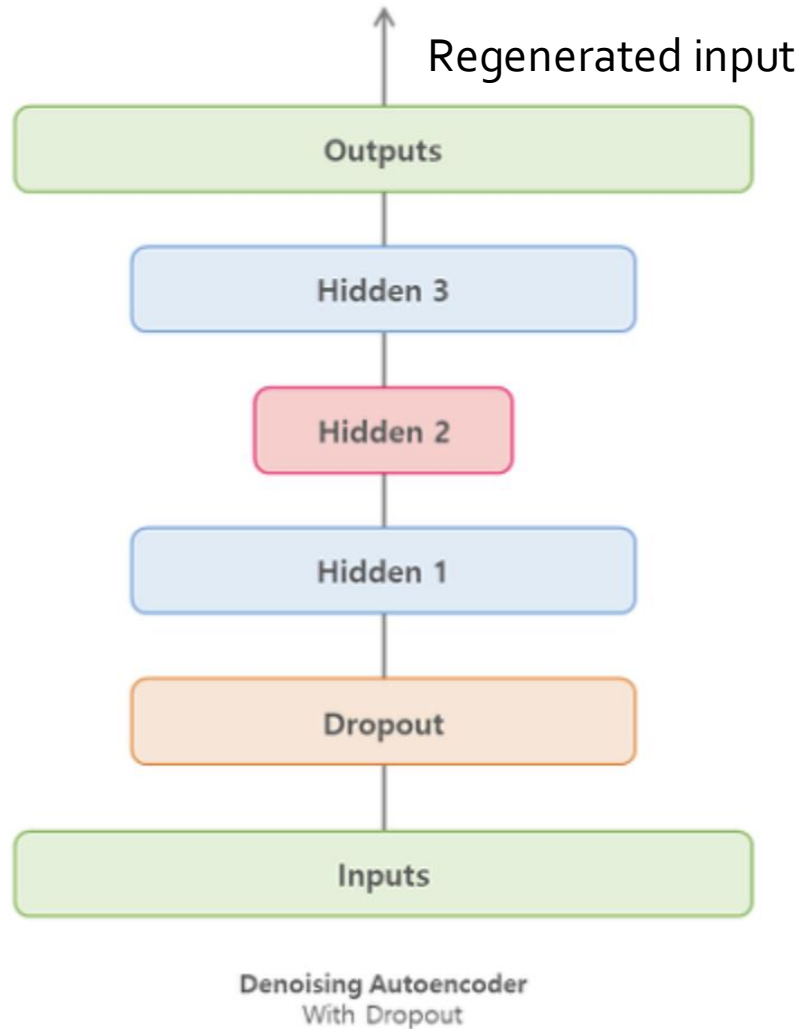


# Regularized autoencoder: Denoising autoencoder w/ Gaussian noise

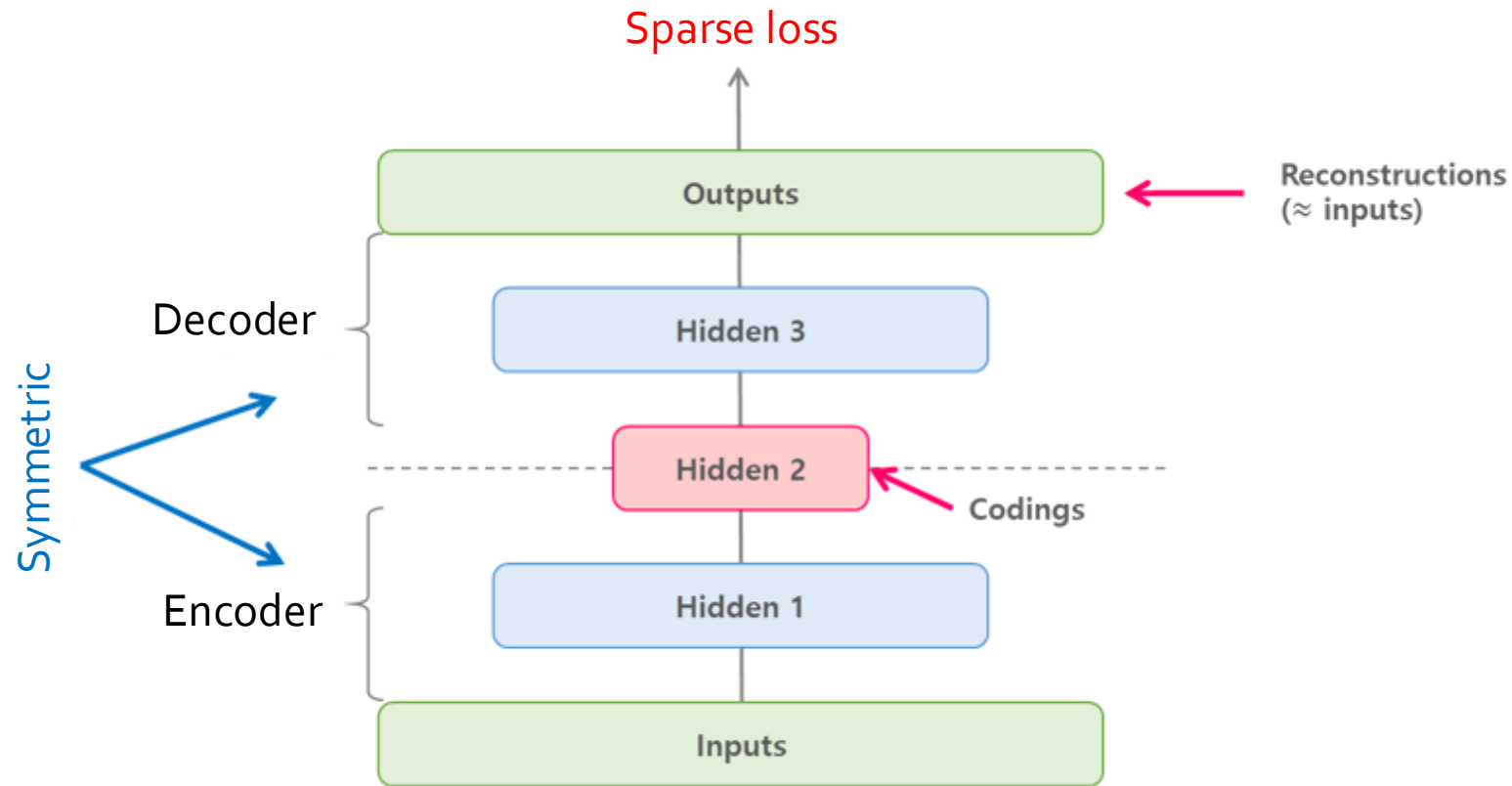


Denoising Autoencoder  
With Gaussian noise

# Regularized autoencoder: Denoising autoencoder w/ Dropout

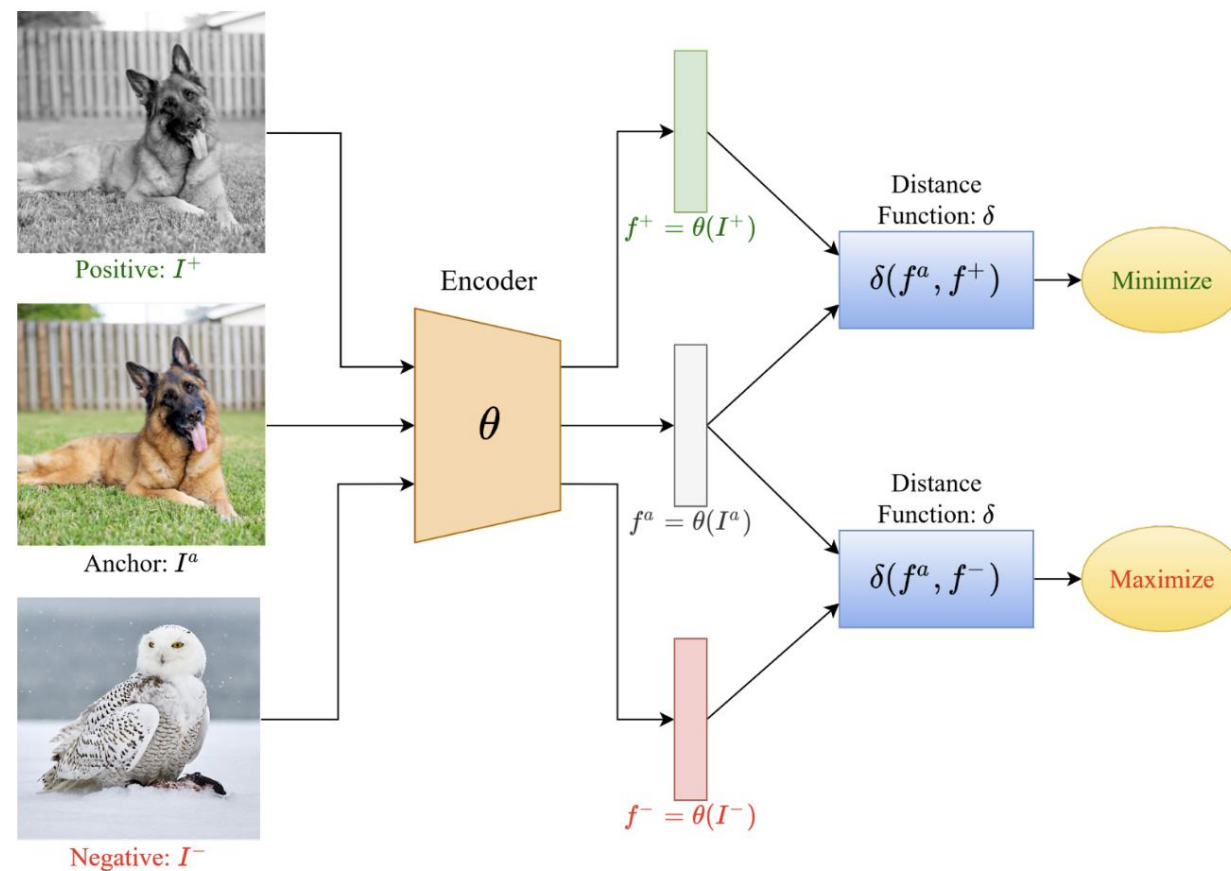


# Regularized autoencoder: Sparse auto encoder



# Contrastive learning

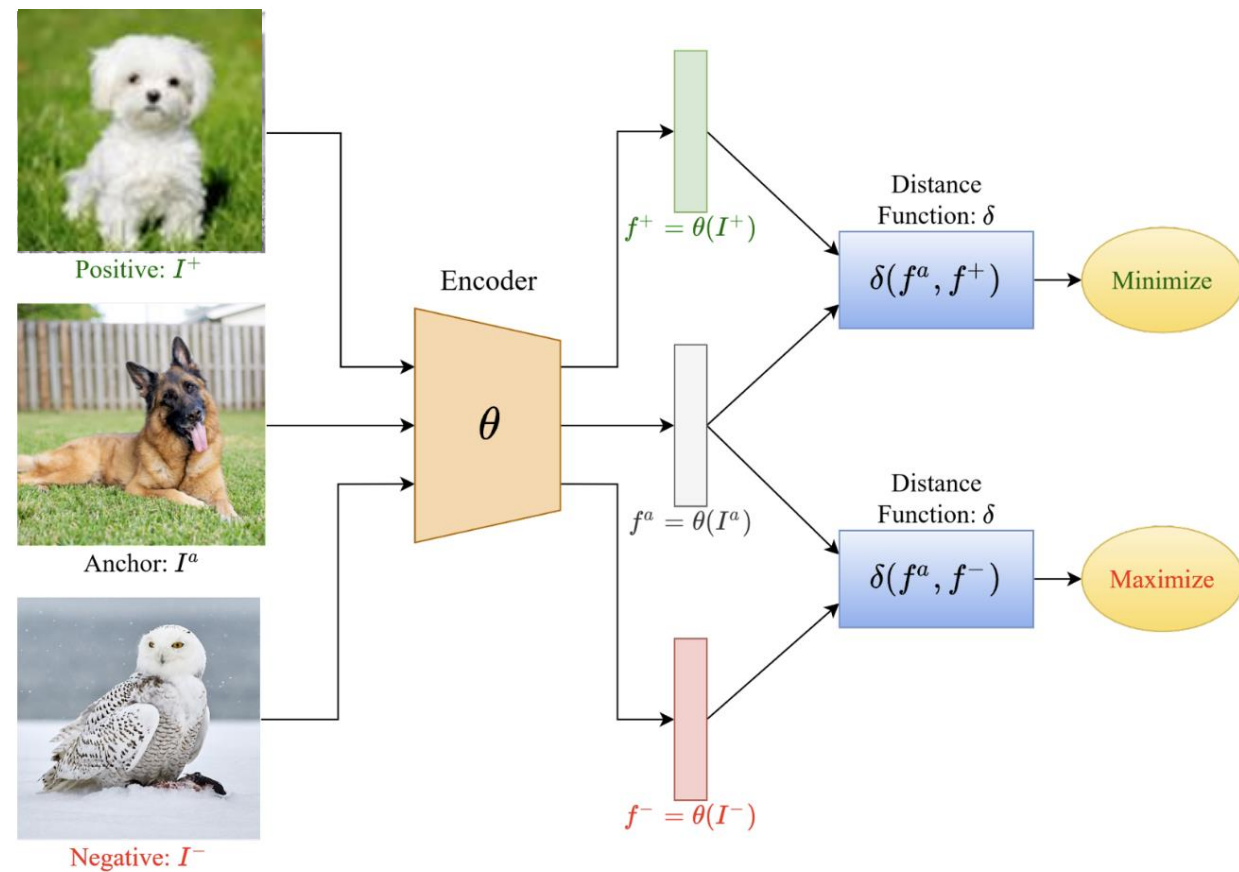
- Unsupervised contrastive learning
  - original





# Contrastive learning

- Supervised contrastive learning
  - Extension



# CLIP

- Developed by OpenAI
- Bridges the gap between computer vision and natural language processing.
- It learns to associate images and text using a **contrastive learning**
  - Enabling it to perform tasks like zero-shot classification without task-specific fine-tuning

# CLIP

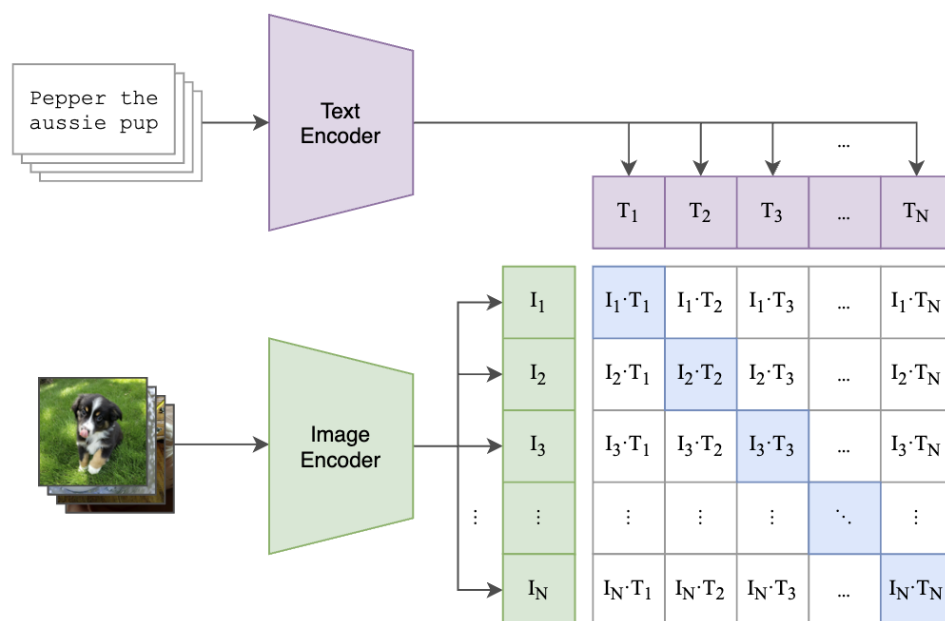


Alec Radford

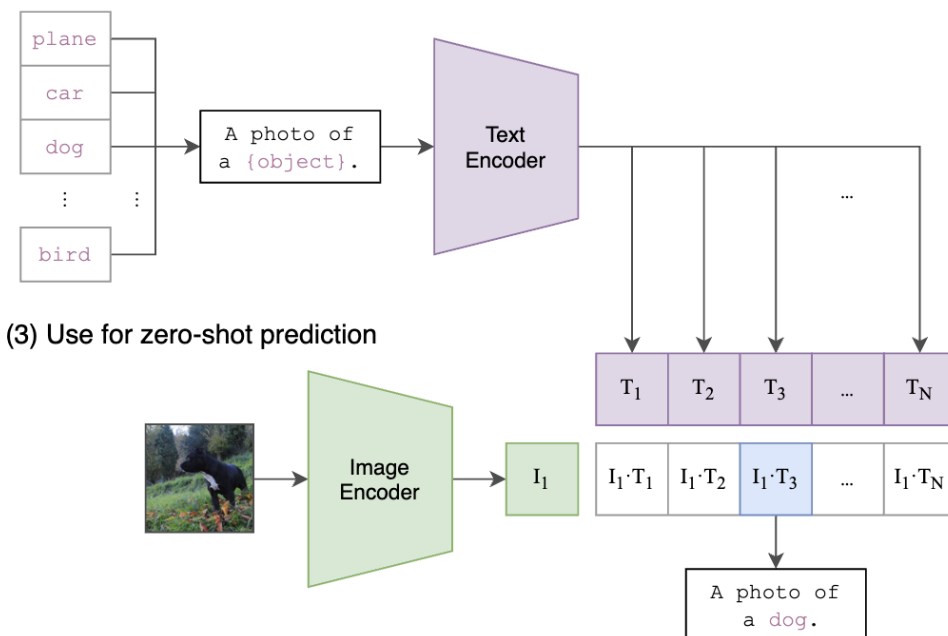


Jongwook Kim

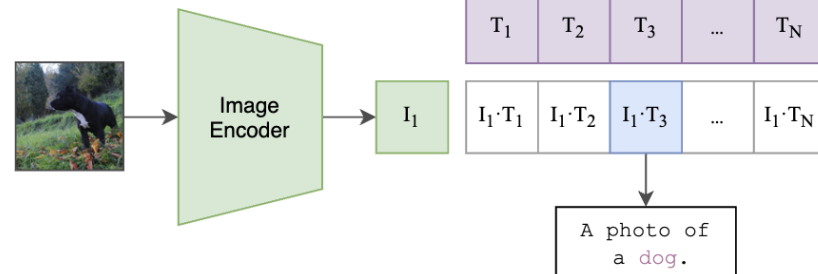
## (1) Contrastive pre-training



## (2) Create dataset classifier from label text



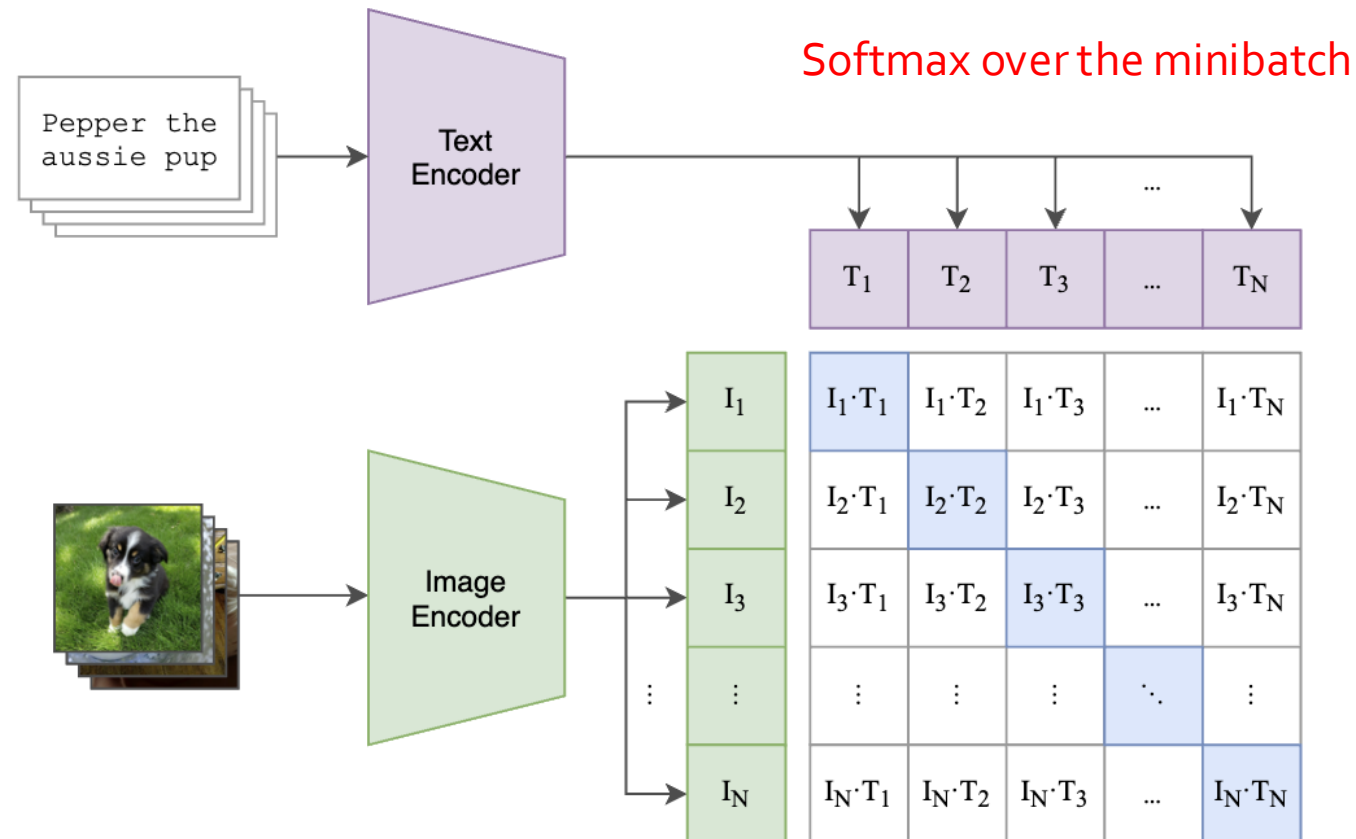
## (3) Use for zero-shot prediction



*Figure 1.* Summary of our approach. While standard image models jointly train an image feature extractor and a linear classifier to predict some label, CLIP jointly trains an image encoder and a text encoder to predict the correct pairings of a batch of (image, text) training examples. At test time the learned text encoder synthesizes a zero-shot linear classifier by embedding the names or descriptions of the target dataset's classes.

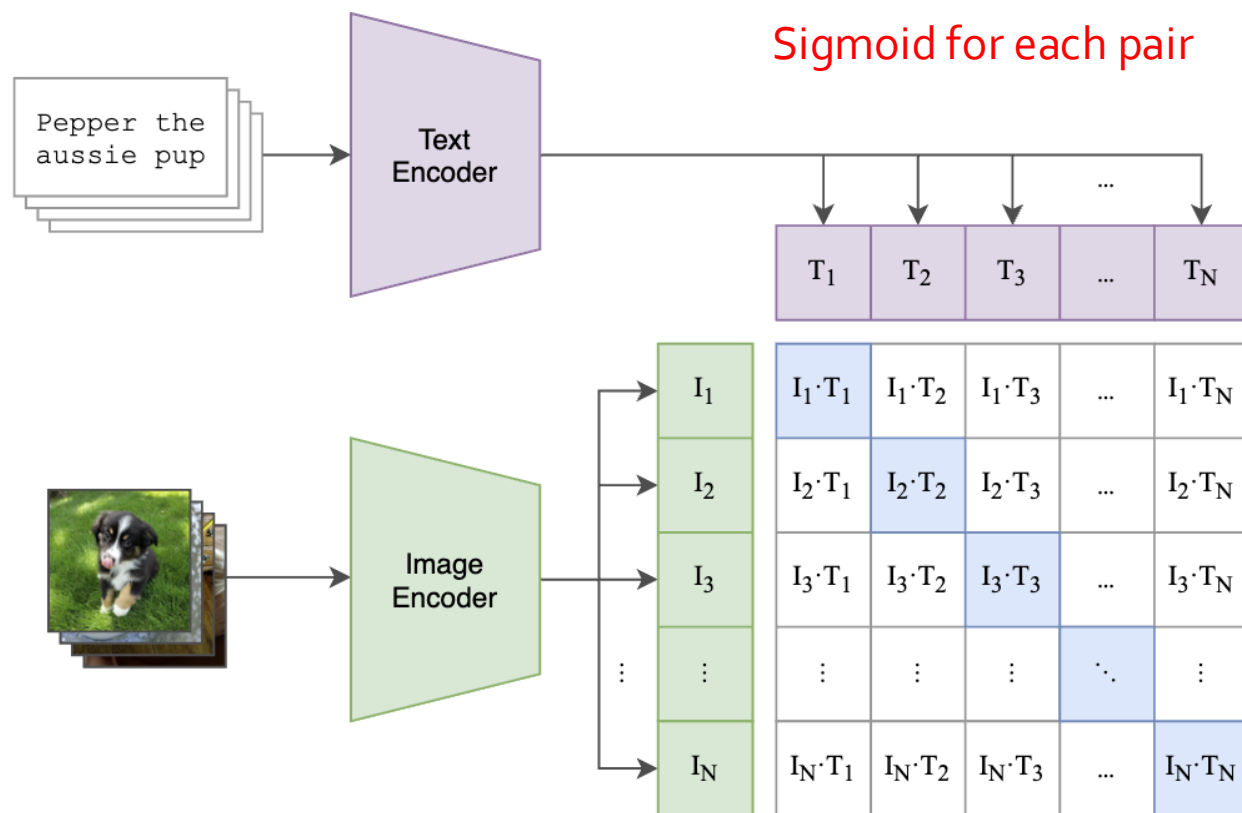
# Contrastive pre-training

## (1) Contrastive pre-training



# SigLIP

## (1) Contrastive pre-training



# Efficient loss implementation using parallelism

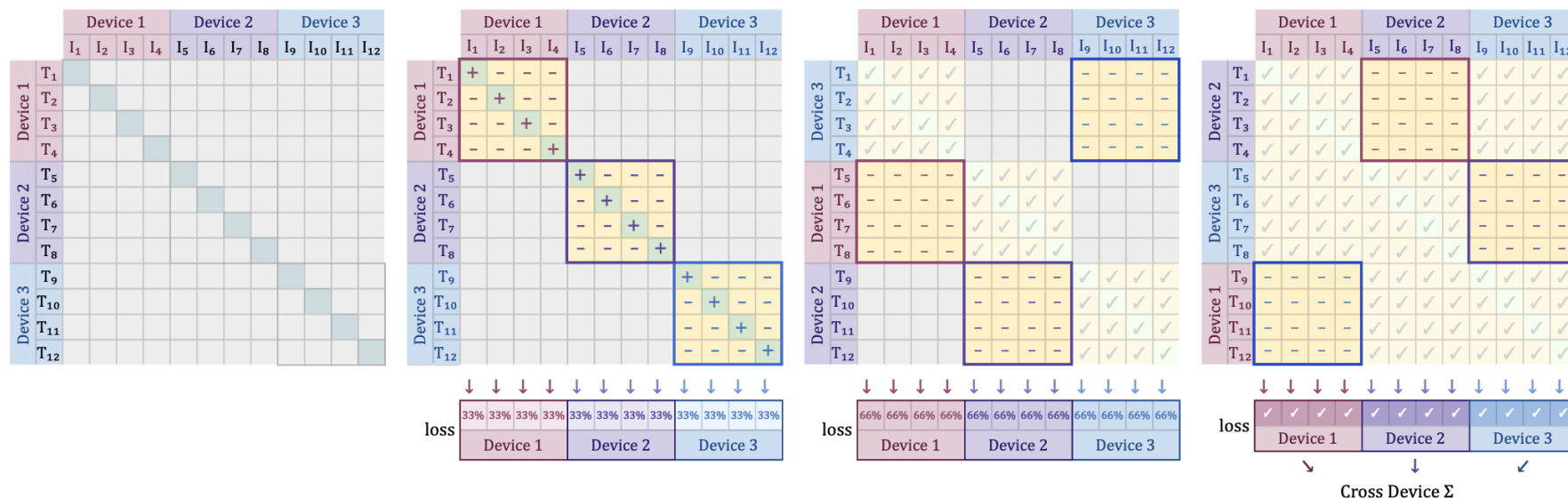


Figure 1: **Efficient loss implementation** demonstrated via a mock setup with 3 devices and a global batch size of 12. There are no all-gathers, and at any point in time only the bright yellow square (size  $4 \times 4$ ) is materialized in memory.

# CLIP vs SigLIP

| Characteristic        | CLIP                             | SigLIP             |
|-----------------------|----------------------------------|--------------------|
| Loss function         | Softmax based contrastive loss   | Sigmoid            |
| On which compute loss | Compare all pairs in a minibatch | Each pair          |
| Memory usage          | Higher                           | Lower              |
| Accuracy              | Fairly good                      | Improved over CLIP |

# CLIP vs SigLIP

| Method   | Image Encoder |           | ImageNet-1k |             |             |             | COCO R@1          |                   |
|----------|---------------|-----------|-------------|-------------|-------------|-------------|-------------------|-------------------|
|          | ViT size      | # Patches | Validation  | v2          | ReaL        | ObjectNet   | I $\rightarrow$ T | T $\rightarrow$ I |
| CLIP     | B             | 196       | 68.3        | 61.9        | -           | 55.3        | 52.4              | 33.1              |
| OpenCLIP | B             | 196       | 70.2        | 62.3        | -           | 56.0        | 59.4              | 42.3              |
| EVA-CLIP | B             | 196       | 74.7        | 67.0        | -           | 62.3        | 58.7              | 42.2              |
| SigLIP   | B             | 196       | <b>76.2</b> | <b>69.6</b> | 82.8        | <b>70.7</b> | <b>64.4</b>       | <b>47.2</b>       |
| SigLIP   | B             | 256       | 76.7        | 70.0        | 83.1        | 71.3        | 65.1              | 47.4              |
| SigLIP   | B             | 576       | 78.6        | 72.1        | 84.5        | 73.8        | 67.5              | 49.7              |
| SigLIP   | B             | 1024      | <b>79.2</b> | <b>73.0</b> | <b>84.9</b> | <b>74.7</b> | <b>67.6</b>       | <b>50.4</b>       |
| CLIP     | L             | 256       | 75.5        | 69.0        | -           | 69.9        | 56.3              | 36.5              |
| OpenCLIP | L             | 256       | 74.0        | 61.1        | -           | 66.4        | 62.1              | 46.1              |
| CLIPA-v2 | L             | 256       | 79.7        | 72.8        | -           | 71.1        | 64.1              | 46.3              |
| EVA-CLIP | L             | 256       | 79.8        | 72.9        | -           | 75.3        | 63.7              | 47.5              |
| SigLIP   | L             | 256       | <b>80.5</b> | <b>74.2</b> | <b>85.9</b> | <b>77.9</b> | <b>69.5</b>       | <b>51.1</b>       |
| CLIP     | L             | 576       | 76.6        | 72.0        | -           | 70.9        | 57.9              | 37.1              |
| CLIPA-v2 | L             | 576       | 80.3        | 73.5        | -           | 73.1        | 65.5              | 47.2              |
| EVA-CLIP | L             | 576       | 80.4        | 73.8        | -           | 78.4        | 64.1              | 47.9              |
| SigLIP   | L             | 576       | <b>82.1</b> | <b>75.9</b> | <b>87.0</b> | <b>81.0</b> | <b>70.6</b>       | <b>52.7</b>       |
| OpenCLIP | G (2B)        | 256       | 80.1        | 73.6        | -           | 73.0        | 67.3              | 51.4              |
| CLIPA-v2 | H (630M)      | 576       | 81.8        | 75.6        | -           | 77.4        | 67.2              | 49.2              |
| EVA-CLIP | E (5B)        | 256       | 82.0        | 75.7        | -           | 79.6        | 68.8              | 51.1              |
| SigLIP   | SO (400M)     | 729       | <b>83.2</b> | <b>77.2</b> | <b>87.5</b> | <b>82.9</b> | <b>70.2</b>       | <b>52.0</b>       |

Table 3: **Comparison with other publicly released models.** Our SigLIP models outperform all prior models, e.g. OpenCLIP [22] and CLIP [36], by a significant margin on both zero-shot classification and retrieval tasks. Compared to the concurrent EVA-CLIP [43] and CLIPA-v2 [29], our SigLIP-L performs better across the board, in both the low and high resolution cases. Especially noteworthy is the Shape-Optimized 400M parameter ViT [1] architecture, which outperforms all significantly larger models. We publicly release our models: [https://github.com/google-research/big\\_vision](https://github.com/google-research/big_vision).



# SigLIP 2

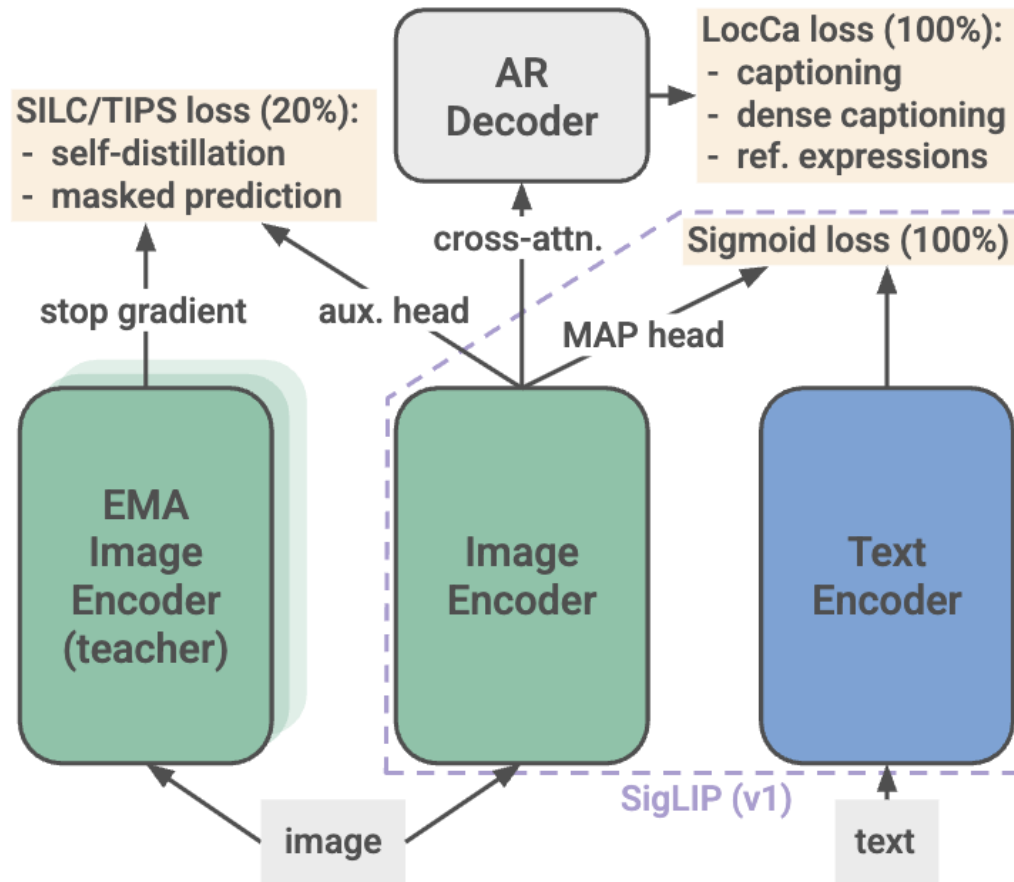
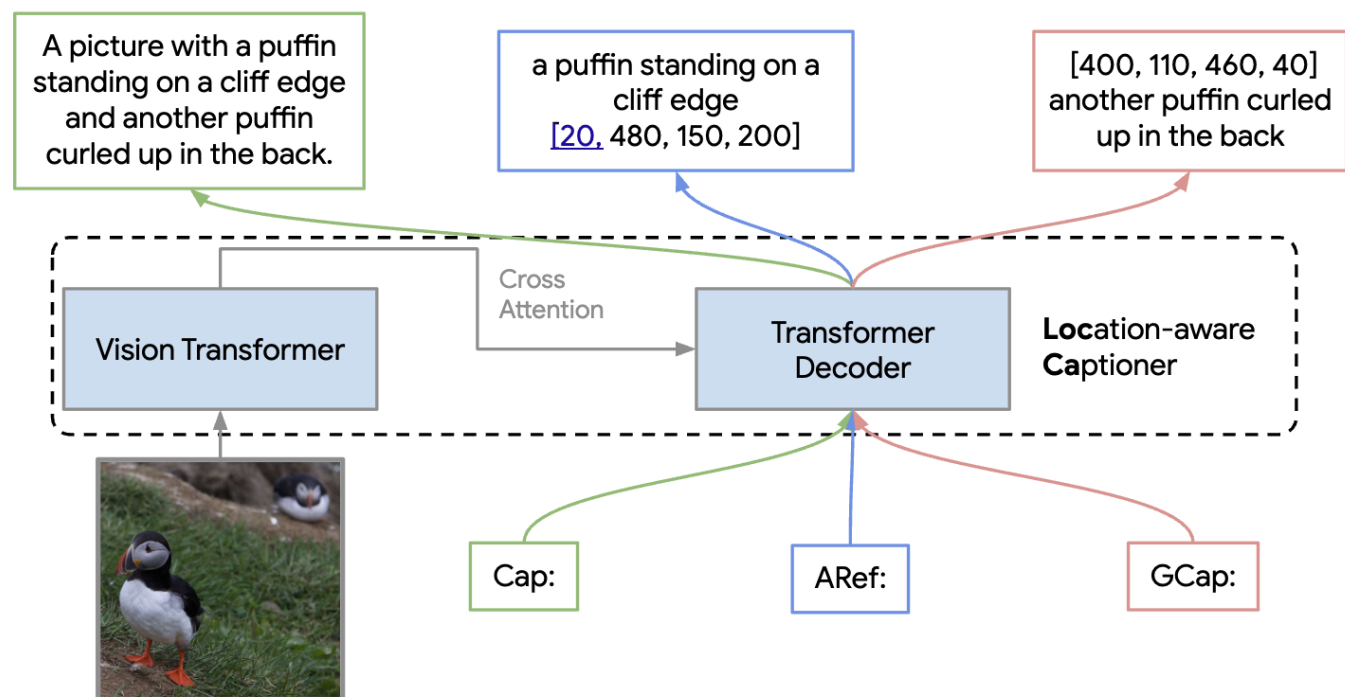


Figure 1 | SigLIP 2 adds the captioning-based pretraining from LocCa [62] as well as self-distillation and masked prediction from SILC [45] and TIPS [38] (during the last 20% of training) to the sigmoid loss from SigLIP [71]. For some variants, the recipe additionally involves fine-tuning with data curation [61] or adaptation to native aspect ratio and variable sequence length [6, 12].

# LocCa



- **Cap:** Captioning
- **ARef:** Automatic referring expression
- **GCap:** Grounded captioning

Figure 1: **Overview of LocCa.** LocCa consists of a standard vision transformer and a transformer decoder. The vision transformer takes image pixel as input, produces visual tokens as cross attention input to the transformer decoder. The transformer decoder is trained to read out rich information from the visual tokens. We adopt the following three task for pretraining: Cap, AREF and GCAP.

# CLIP vs SigLIP vs SigLIP 2

| Characteristic        | CLIP                             | SigLIP              | SigLIP 2                                     |
|-----------------------|----------------------------------|---------------------|----------------------------------------------|
| Loss function         | Softmax based contrastive loss   | Sigmoid             | LocCa                                        |
| Task                  | Image-Text matching              | Image-Text matching | Captioning, Distillation                     |
| On which compute loss | Compare all pairs in a minibatch | Each pair           | Each pair                                    |
| Memory usage          | Higher                           | Lower               | Lower                                        |
| Accuracy              | Fairly good                      | Improved over CLIP  | Improved over SigLIP in many different tasks |

# CLIP vs SigLIP vs SigLIP 2

| ViT   | Res. | Seq. | Model         | ImageNet-1k |      |      |        |      | COCO |      | Flickr |      | XM3600 |      |
|-------|------|------|---------------|-------------|------|------|--------|------|------|------|--------|------|--------|------|
|       |      |      |               | val         | v2   | ReaL | ObjNet | 10s. | T→I  | I→T  | T→I    | I→T  | T→I    | I→T  |
| B/32  | 224  | 49   | MetaCLIP [66] | 67.7        | 59.6 | –    | 52.8   | –    | 46.6 | –    | 72.9   | –    | –      | –    |
|       | 256  | 64   | OpenCLIP [27] | 72.8        | 64.8 | –    | 59.6   | –    | 39.9 | 57.9 | 64.9   | 84.8 | –      | –    |
|       |      |      | SigLIP 2      | 74.0        | 66.9 | 81.4 | 66.1   | 66.6 | 47.2 | 63.7 | 75.5   | 89.3 | 38.3   | 49.0 |
| B/16  | 224  | 196  | CLIP [50]     | 68.3        | 61.9 | –    | 55.3   | –    | 33.1 | 52.4 | 62.1   | 81.9 | –      | –    |
|       |      |      | OpenCLIP [27] | 70.2        | 62.3 | –    | 56.0   | –    | 42.3 | 59.4 | 69.8   | 86.3 | –      | –    |
|       |      |      | MetaCLIP [66] | 72.4        | 65.1 | –    | 60.0   | –    | 48.9 | –    | 77.1   | –    | –      | –    |
|       |      |      | EVA-CLIP [57] | 74.7        | 67.0 | –    | 62.3   | –    | 42.2 | 58.7 | 71.2   | 85.7 | –      | –    |
|       |      |      | SigLIP [71]   | 76.2        | 69.5 | 82.8 | 70.7   | 69.9 | 47.2 | 64.5 | 77.9   | 89.6 | 22.4   | 29.3 |
|       |      |      | DFN [19]      | 76.2        | 68.2 | –    | 63.2   | –    | 51.9 | –    | 77.3   | –    | –      | –    |
|       |      |      | SigLIP 2      | 78.2        | 71.4 | 84.8 | 73.6   | 72.1 | 52.1 | 68.9 | 80.7   | 93.0 | 40.3   | 50.7 |
|       | 256  | 256  | SigLIP [71]   | 76.7        | 70.1 | 83.1 | 71.3   | 70.3 | 47.4 | 65.1 | 78.3   | 91.1 | 22.5   | 29.9 |
|       |      |      | SigLIP 2      | 79.1        | 72.5 | 85.4 | 74.5   | 73.1 | 53.2 | 69.7 | 81.7   | 94.4 | 40.7   | 51.0 |
|       | 384  | 576  | SigLIP [71]   | 78.6        | 72.0 | 84.6 | 73.8   | 72.7 | 49.7 | 67.5 | 80.7   | 92.2 | 23.3   | 30.3 |
|       |      |      | SigLIP 2      | 80.6        | 73.8 | 86.2 | 77.1   | 74.7 | 54.6 | 71.4 | 83.8   | 94.9 | 41.2   | 51.6 |
|       | 512  | 1024 | SigLIP [71]   | 79.2        | 72.9 | 84.9 | 74.8   | 73.3 | 50.4 | 67.6 | 81.6   | 92.5 | 23.5   | 30.5 |
|       |      |      | SigLIP 2      | 81.2        | 74.5 | 86.7 | 77.8   | 75.2 | 55.2 | 71.2 | 84.5   | 95.5 | 41.4   | 52.0 |
| L/14  | 224  | 256  | OpenCLIP [27] | 74.0        | 61.1 | –    | 66.4   | –    | 46.1 | 62.1 | 75.0   | 88.7 | –      | –    |
|       |      |      | CLIP [50]     | 75.5        | 69.0 | –    | 69.9   | –    | 36.5 | 56.3 | 65.2   | 85.2 | –      | –    |
|       |      |      | MetaCLIP [66] | 79.2        | 72.6 | –    | 74.6   | –    | 55.7 | –    | 83.3   | –    | –      | –    |
|       |      |      | CLIPA-v2 [33] | 79.7        | 72.8 | –    | 71.1   | –    | 46.3 | 64.1 | 73.0   | 89.1 | –      | –    |
|       |      |      | EVA-CLIP [57] | 79.8        | 72.9 | –    | 75.3   | –    | 47.5 | 63.7 | 77.3   | 89.7 | –      | –    |
|       |      |      | DFN [19]      | 82.2        | 75.7 | –    | 74.8   | –    | 59.6 | –    | 84.7   | –    | –      | –    |
| L/16  | 256  | 256  | SigLIP [71]   | 80.5        | 74.2 | 85.9 | 77.9   | 76.8 | 51.2 | 69.6 | 81.3   | 92.0 | 30.9   | 40.1 |
|       |      |      | SigLIP 2      | 82.5        | 76.8 | 87.3 | 83.0   | 78.8 | 54.7 | 71.5 | 84.1   | 94.5 | 46.5   | 56.5 |
|       | 384  | 576  | SigLIP [71]   | 82.1        | 75.9 | 87.1 | 80.9   | 78.7 | 52.8 | 70.5 | 82.6   | 92.9 | 31.4   | 39.7 |
|       |      |      | SigLIP 2      | 83.1        | 77.4 | 87.6 | 84.4   | 79.5 | 55.3 | 71.4 | 85.0   | 95.2 | 47.1   | 56.3 |
| So/14 | 224  | 256  | SigLIP [71]   | 82.2        | 76.0 | 87.1 | 80.5   | 78.2 | 50.8 | 69.0 | 76.6   | 90.7 | 16.0   | 22.8 |
|       |      |      | SigLIP 2      | 83.2        | 77.7 | 87.8 | 84.6   | 79.5 | 55.1 | 71.5 | 84.3   | 94.6 | 47.9   | 57.5 |
|       | 384  | 729  | SigLIP [71]   | 83.2        | 77.1 | 87.5 | 82.9   | 79.4 | 52.0 | 70.2 | 80.5   | 93.5 | 17.8   | 26.6 |
|       |      |      | SigLIP 2      | 84.1        | 78.7 | 88.1 | 86.0   | 80.4 | 55.8 | 71.7 | 85.7   | 94.9 | 48.4   | 57.5 |
| So/16 | 256  | 256  | mSigLIP [71]  | 80.8        | 74.1 | 86.1 | 79.5   | 77.1 | 49.4 | 68.6 | 80.0   | 92.1 | 50.0   | 62.8 |
|       |      |      | SigLIP 2      | 83.4        | 77.8 | 87.7 | 84.8   | 79.7 | 55.4 | 71.5 | 84.4   | 94.2 | 48.1   | 57.5 |
|       | 384  | 576  | SigLIP 2      | 84.1        | 78.4 | 88.1 | 85.8   | 80.4 | 56.0 | 71.2 | 85.3   | 95.9 | 48.3   | 57.5 |
|       |      |      | SigLIP 2      | 84.3        | 79.1 | 88.1 | 86.2   | 80.5 | 56.0 | 71.3 | 85.5   | 95.4 | 48.3   | 57.6 |
| H/14  | 224  | 256  | MetaCLIP [66] | 80.5        | 74.1 | –    | 76.5   | –    | 57.5 | –    | 85.0   | –    | –      | –    |
|       |      |      | DFN [19]      | 83.4        | 77.3 | –    | 76.5   | –    | 63.1 | –    | 86.5   | –    | –      | –    |
| g/16  | 256  | 256  | SigLIP 2      | 84.5        | 79.2 | 88.3 | 87.1   | 82.1 | 55.7 | 72.5 | 85.3   | 95.3 | 48.2   | 58.2 |
|       | 384  | 576  | SigLIP 2      | 85.0        | 79.8 | 88.5 | 88.0   | 82.5 | 56.1 | 72.8 | 86.0   | 95.4 | 48.6   | 57.9 |

Table 1 | Zero-shot classification, 10-shot (10s) classification (on the validation set), and retrieval performance (recall@1) of SigLIP 2 along with several baselines. SigLIP 2 outperforms the baselines—often by a large margin—despite being multilingual. Note that DFN [19] relies on a data filtering network fine-tuned on ImageNet, COCO, and Flickr.

# Self-supervised by teacher-student framework with a single model

- DINO [1]
  - Visualize the small patch to be labeled as dog



# Self-supervised by teacher-student framework with a single model

- DINO [1]
  - Teacher-student training scheme with a single model

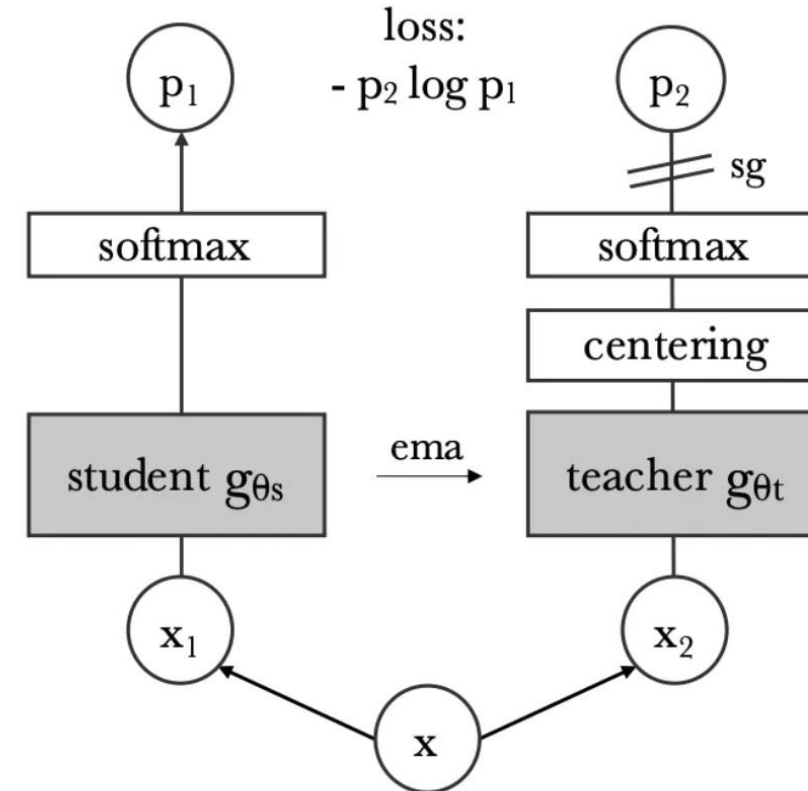


Figure 2: **Self-distillation with no labels.** We illustrate DINO in



# Self-supervised by teacher-student framework with a single model

- DINO [1]
  - Better than supervised attention

*Supervised*



*DINO*



|          | Random | Supervised | DINO |
|----------|--------|------------|------|
| ViT-S/16 | 22.0   | 27.3       | 45.9 |
| ViT-S/8  | 21.8   | 23.7       | 44.7 |

# Pretraining: self-supervised pretext tasks

- In computer vision
  - Learn to better encode 2D information in natural images

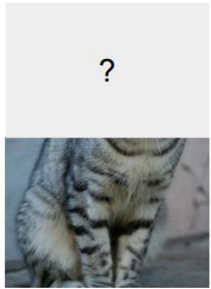
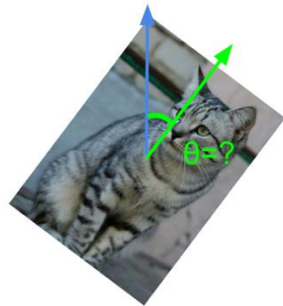


image completion



rotation prediction



“jigsaw puzzle”



colorization

- Natural language processing
  - Learn to better infer the probability of the next word  
→ **Masked language modelling**

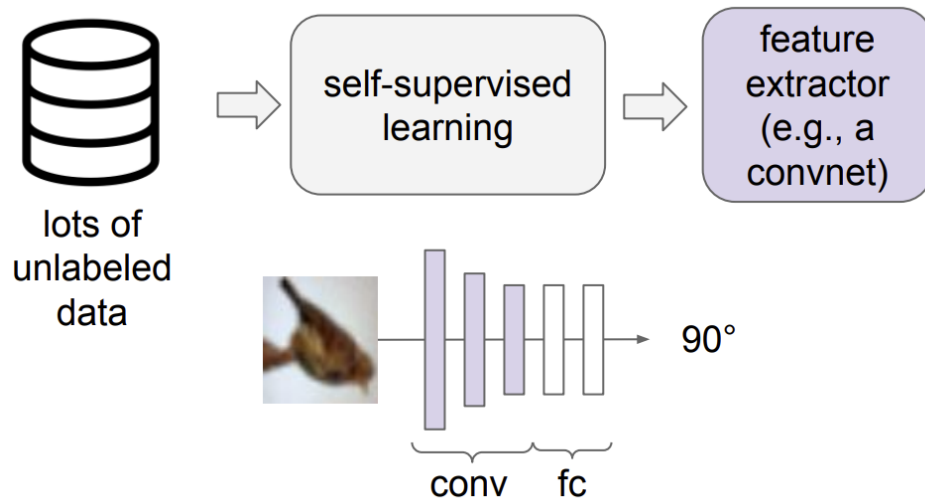


# How to evaluate a self-supervised learning method?

- We usually don't care about the performance of the self-supervised learning task
  - e.g., we don't care if the model learns to predict image rotation perfectly
- Evaluate the learned feature encoders on downstream tasks (target tasks)

# How to evaluate a self-supervised learning method? (cont'd)

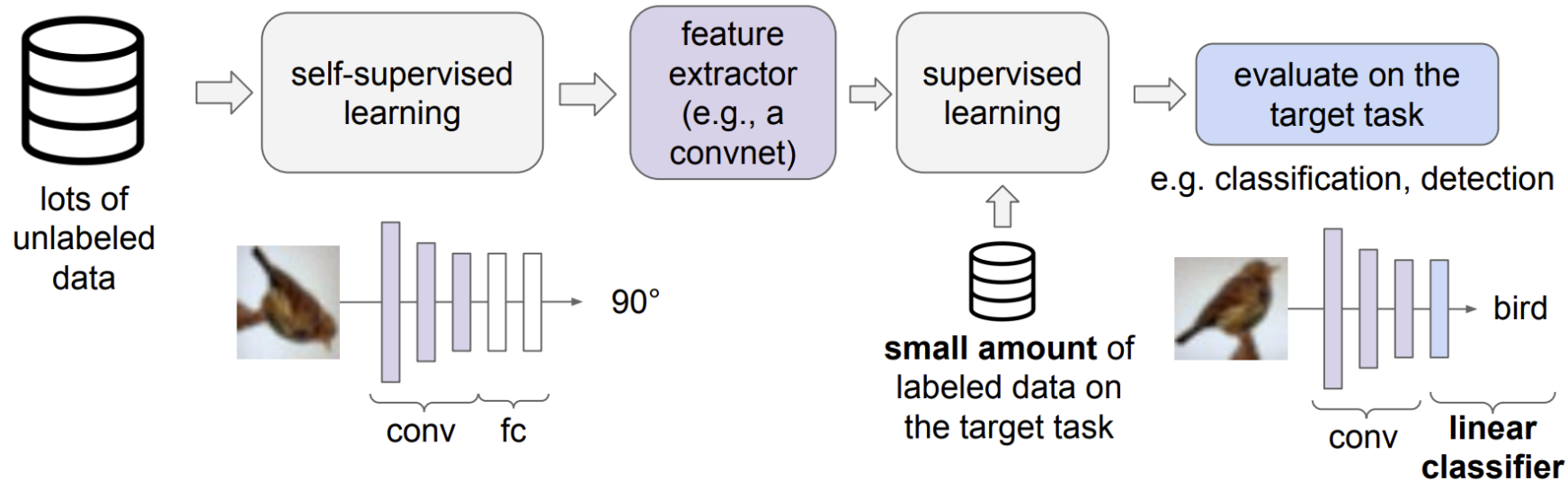
- A slide by Fei-Fei, Jiajun and Ruohan



1. Learn good feature extractors from self-supervised pretext tasks, e.g., predicting image rotations

# How to evaluate a self-supervised learning method? (cont'd)

- A slide by Fei-Fei, Jiajun and Ruohan



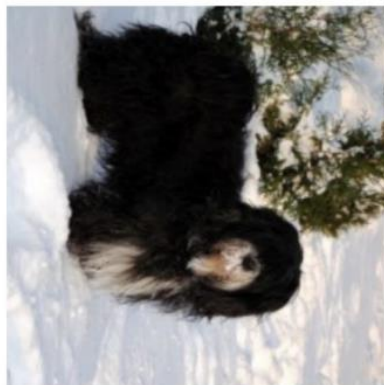
1. Learn good feature extractors from self-supervised pretext tasks, e.g., predicting image rotations

2. Attach a shallow network on the feature extractor; train the shallow network on the target task with small amount of labeled data

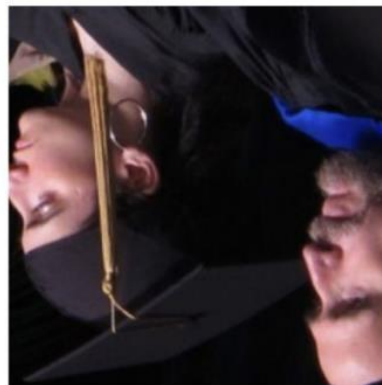
# Pretext task of CV – 1: Predict rotations



90° rotation



270° rotation



180° rotation



0° rotation

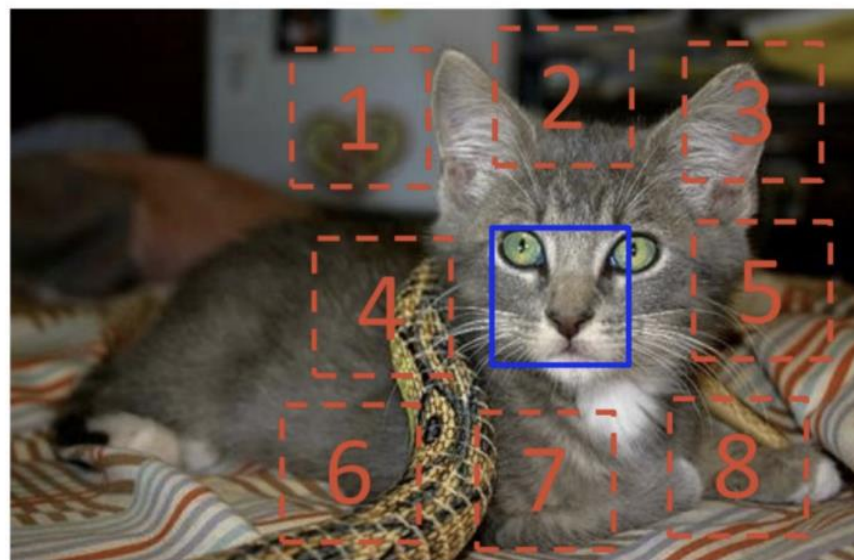


270° rotation

**Hypothesis:** a model could recognize the correct rotation of an object only if it has the “visual commonsense” of what the object should look like unperturbed.

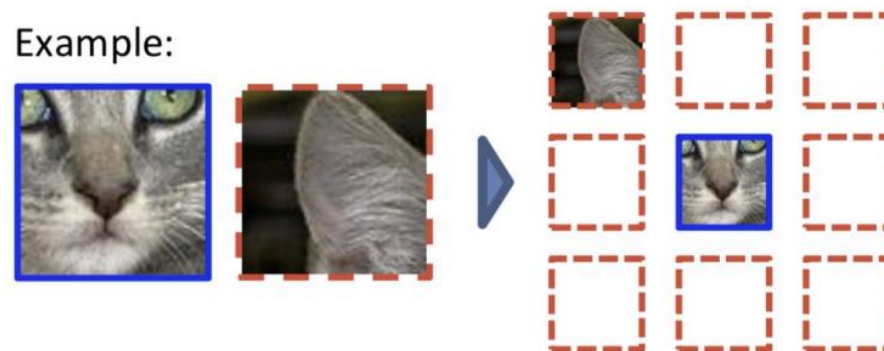
(Image source: [Gidaris et al. 2018](#))

# Pretext task of CV – 2: Predict relative patch locations



$$X = (\text{cat\_eyes}, \text{cat\_ear}); Y = 3$$

Example:



Question 1:



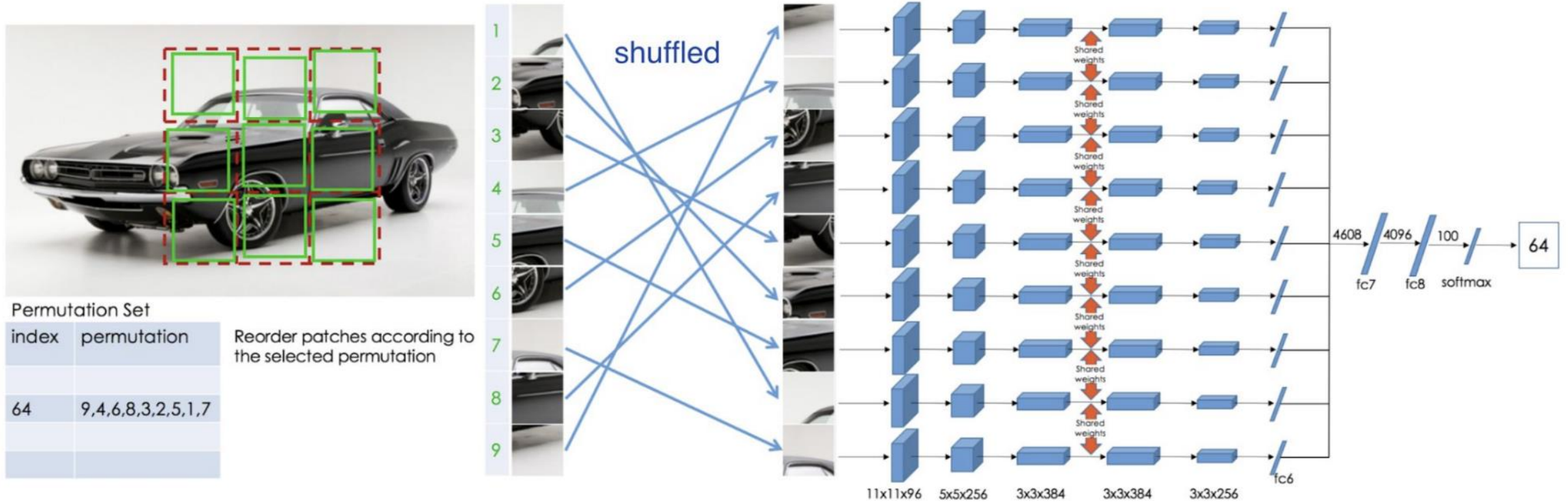
Question 2:



(Image source: [Doersch et al., 2015](#))

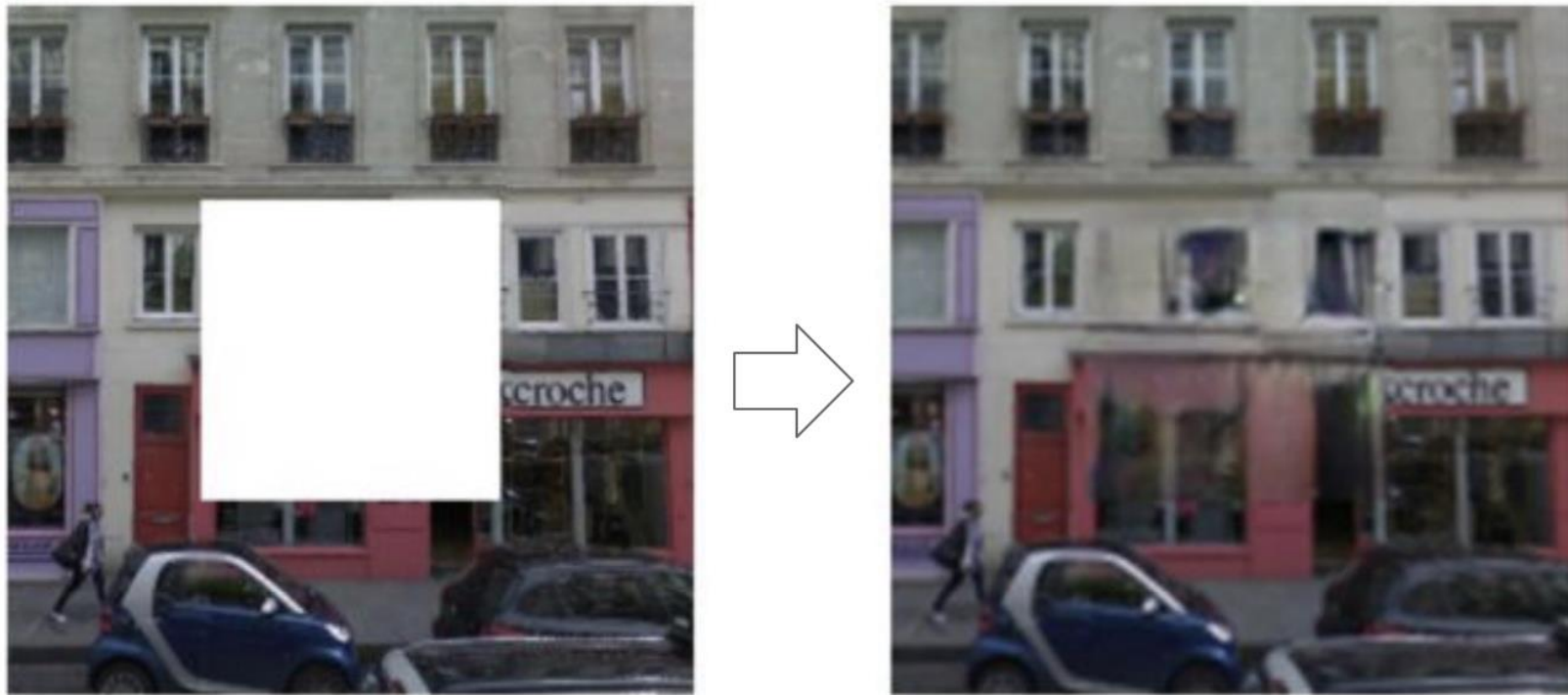


# Pretext task of CV – 3: Solving “jigsaw puzzles”



(Image source: [Noroozi & Favaro, 2016](#))

# Pretext task of CV – 4: Predicting missing pixels (inpainting)

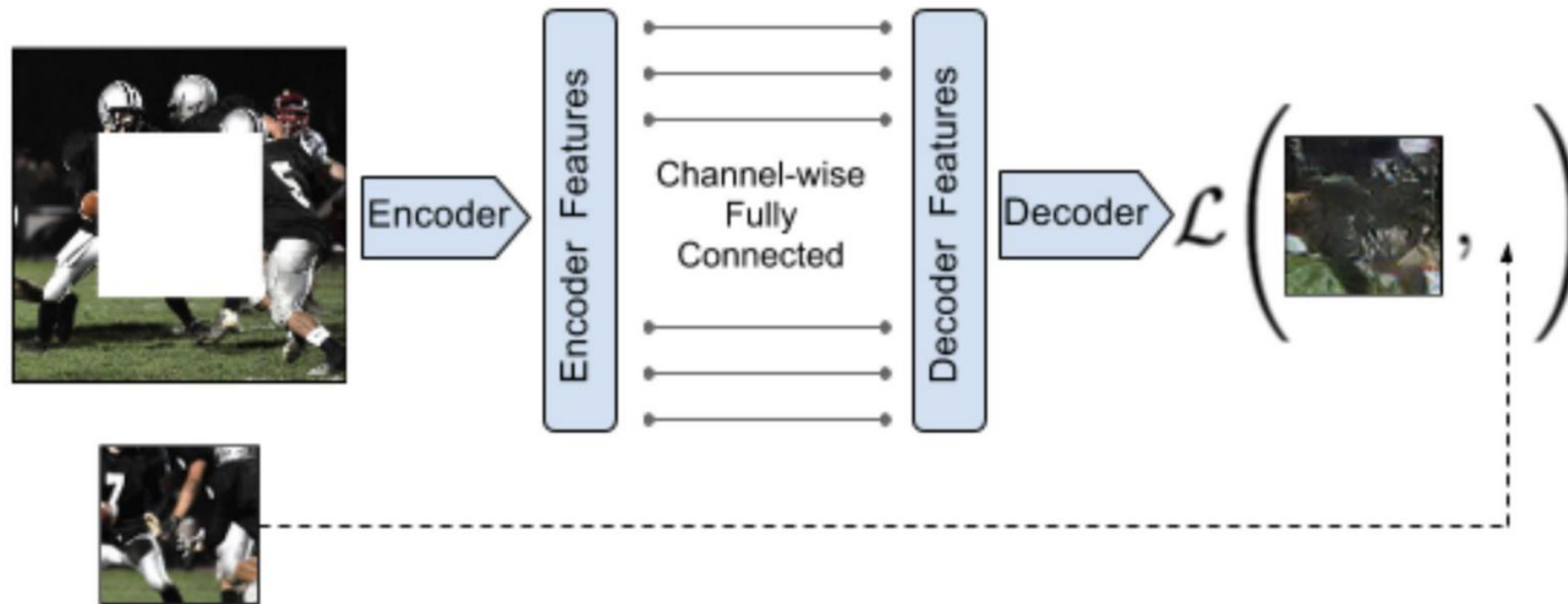


*Context Encoders: Feature Learning by Inpainting* (Pathak et al., 2016)

Source: [Pathak et al., 2016](#)

- A slide by Fei-Fei, Jiajun and Ruohan

# Pretext task of CV – 4: Predicting missing pixels (inpainting) (cont'd)



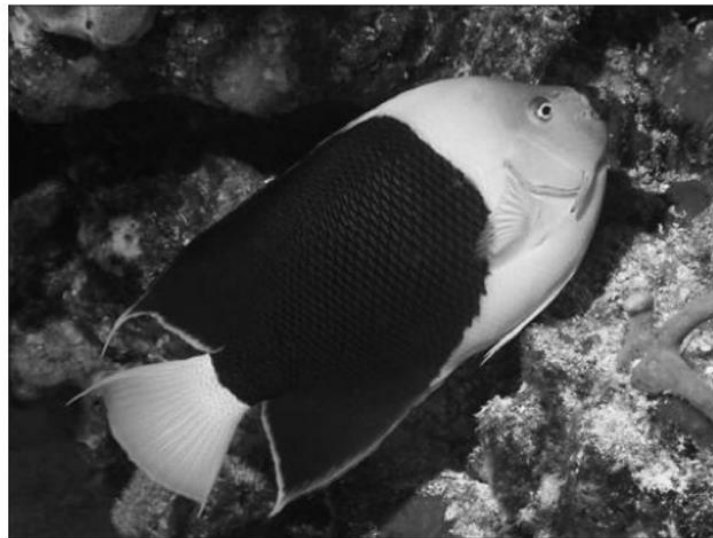
Learning to reconstruct the missing pixels

Source: [Pathak et al., 2016](#)

- A slide by Fei-Fei, Jiajun and Ruohan

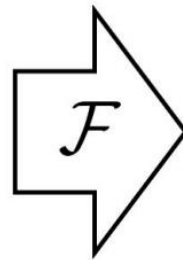


# Pretext task of CV – 5: Predicting color in an image (image coloring)



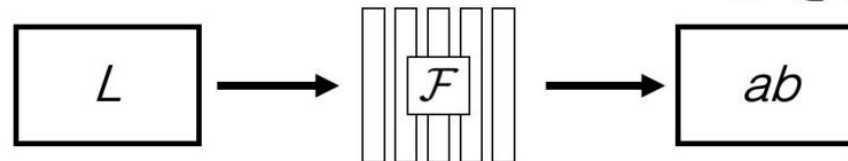
Grayscale image:  $L$  channel

$$\mathbf{X} \in \mathbb{R}^{H \times W \times 1}$$



Color information:  $ab$  channels

$$\hat{\mathbf{Y}} \in \mathbb{R}^{H \times W \times 2}$$



Source: Richard Zhang / Phillip Isola

- A slide by Fei-Fei, Jiajun and Ruohan

# Pretext task of CV – 5: Predicting color in an image (image coloring) (cont'd)



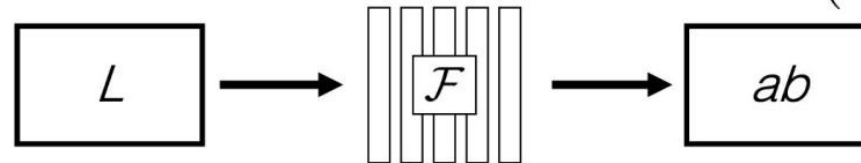
Grayscale image:  $L$  channel

$$\mathbf{X} \in \mathbb{R}^{H \times W \times 1}$$



Concatenate  $(L, ab)$  channels

$$(\mathbf{X}, \hat{\mathbf{Y}})$$



Source: Richard Zhang / Phillip Isola

- A slide by Fei-Fei, Jiajun and Ruohan

# Pretext task of CV – 6: Predicting color in a video (video coloring)

reference frame



$t = 0$

how should I color these frames?



$t = 1$



$t = 2$



$t = 3$

...

Source: [Vondrick et al., 2018](#)

- A slide by Fei-Fei, Jiajun and Ruohan



# Pretext task of CV – 6: Predicting color in a video (video coloring)

reference frame

how should I color these frames?

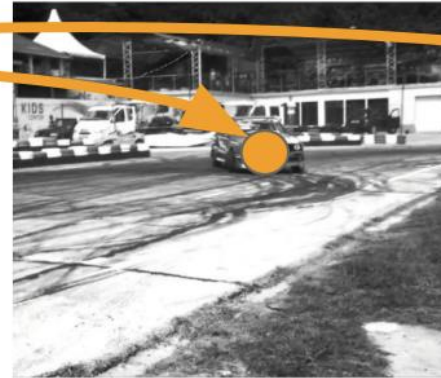
**Should be the same color!**



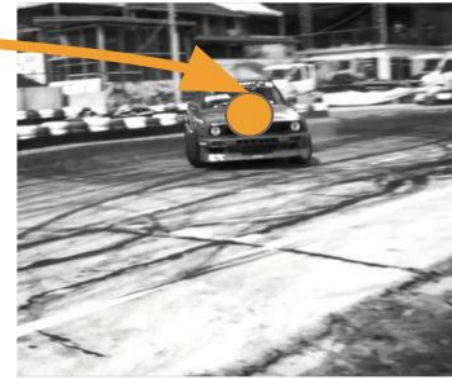
t = 0



t = 1



t = 2



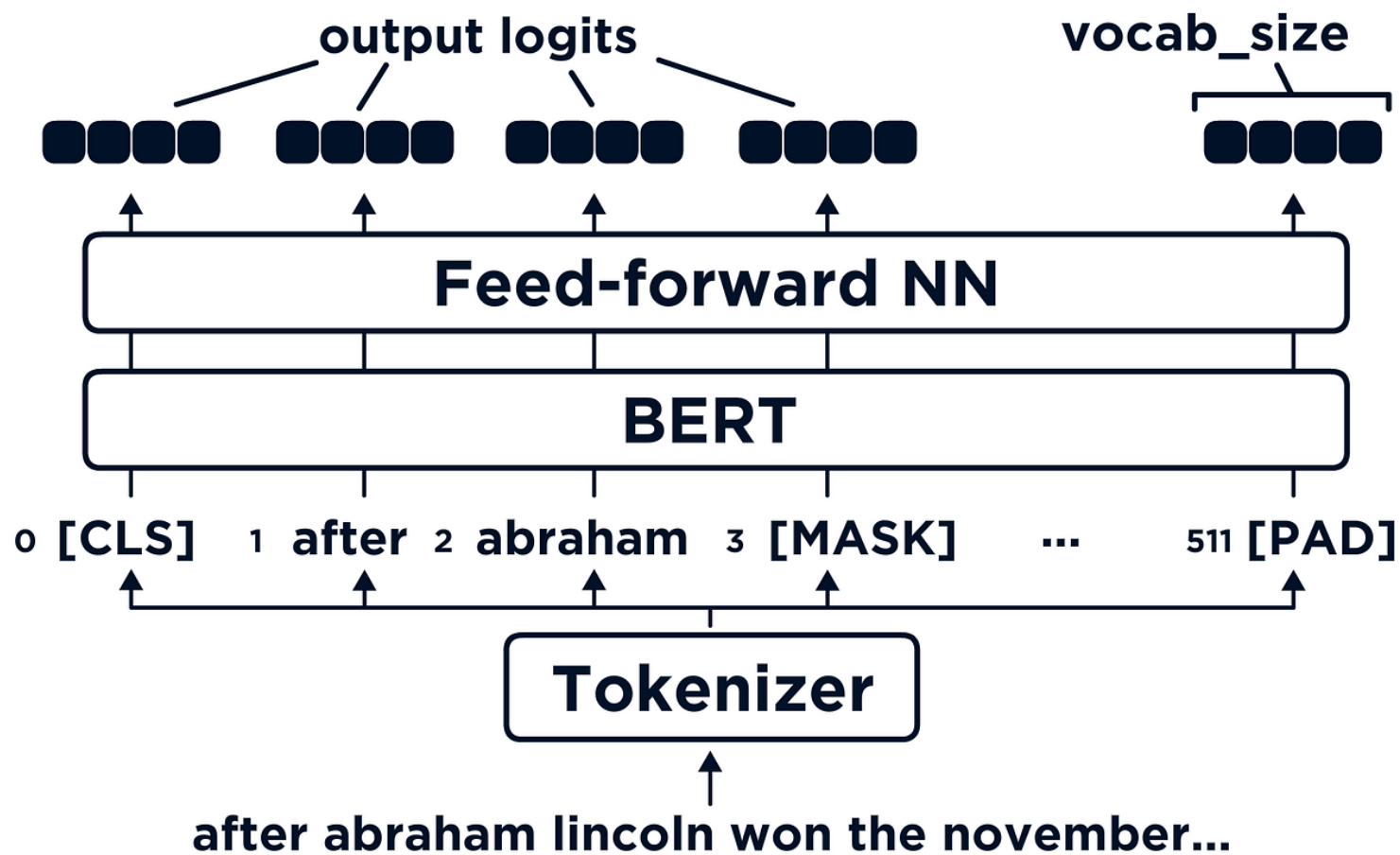
t = 3

...

**Hypothesis:** learning to color video frames should allow model to learn to track regions or objects without labels!

Source: [Vondrick et al., 2018](#)

# Pretext task of NLP: Language model



# Summary

- Self-supervised learning
  - Convolutional layers
  - (max) Pooling layers
  - MLP (fully connected) layers